

Week 06:

Database-backed Websites

COMS 4170 • Spring 2026

Prof. Celeste Layne
Tuesday February 24, 2026

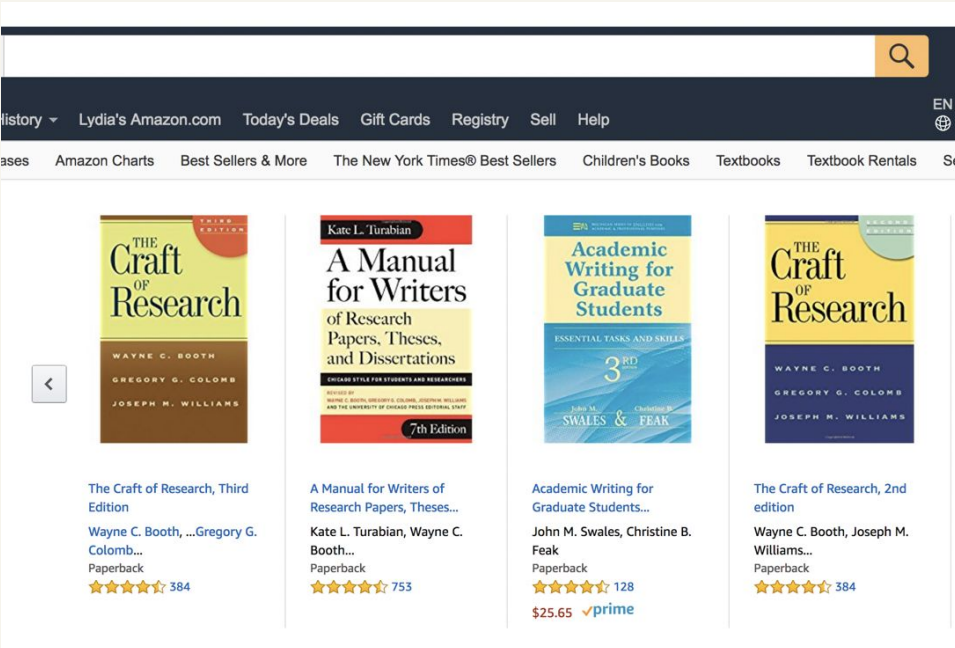
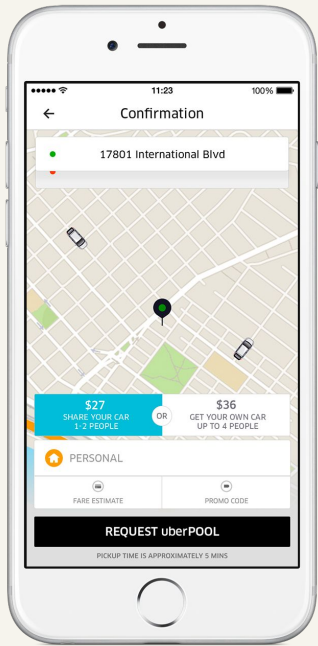
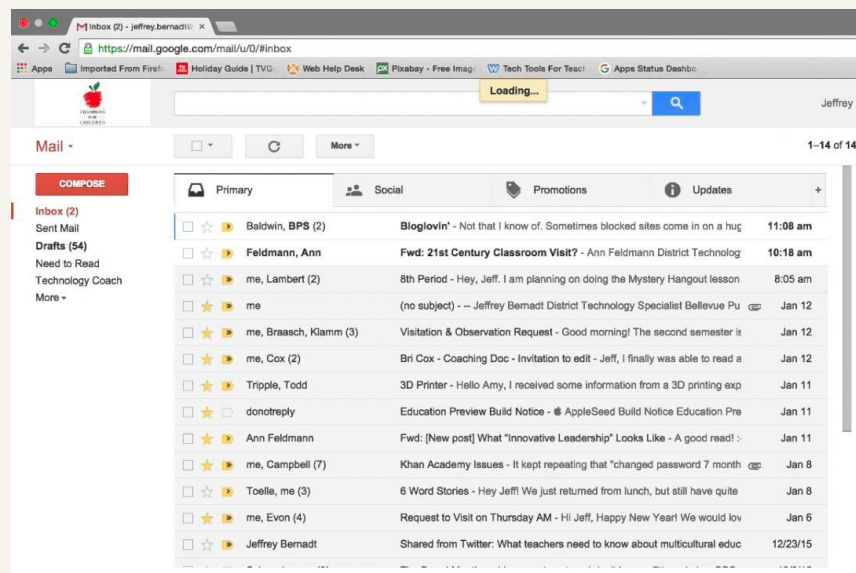


COMS 4170 · SPRING 2026

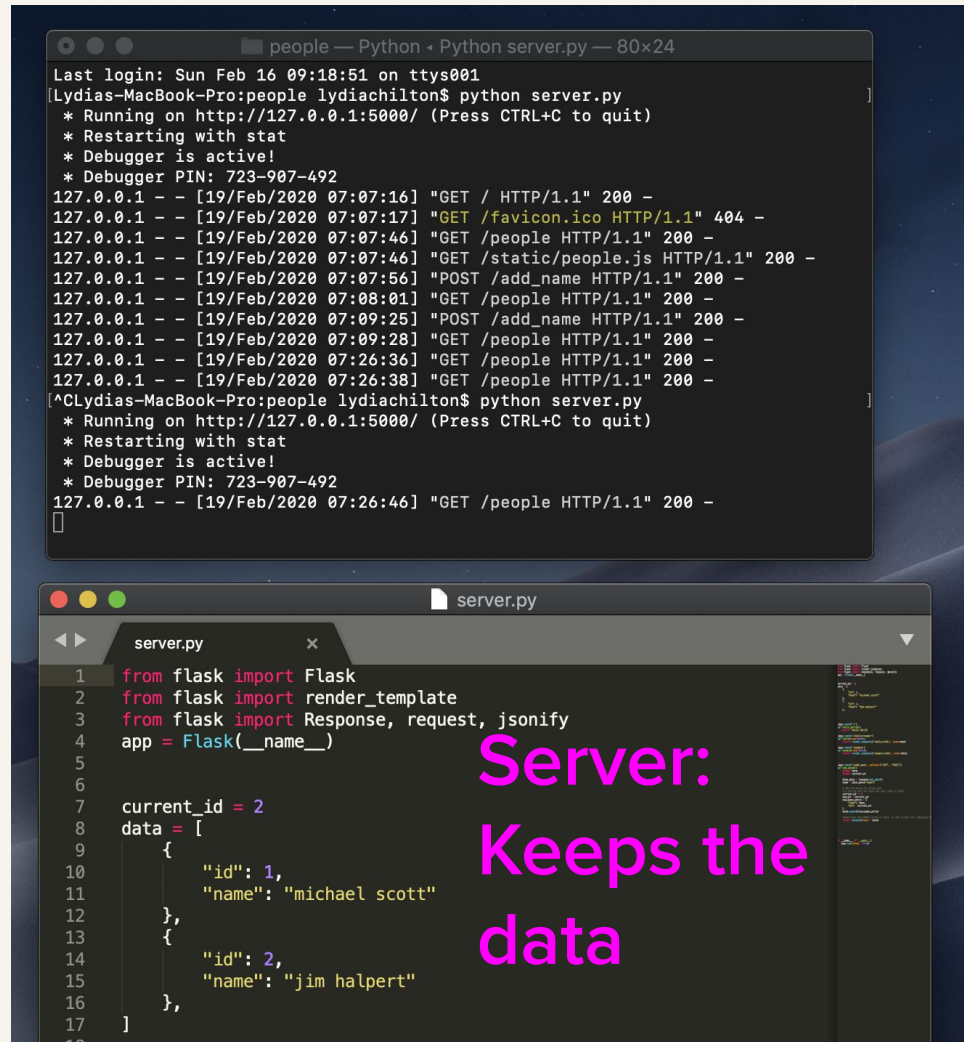
Review



The main goal of many websites is to interact with data.



We need servers to **store the data** and clients to **request the data** from servers and display it to users

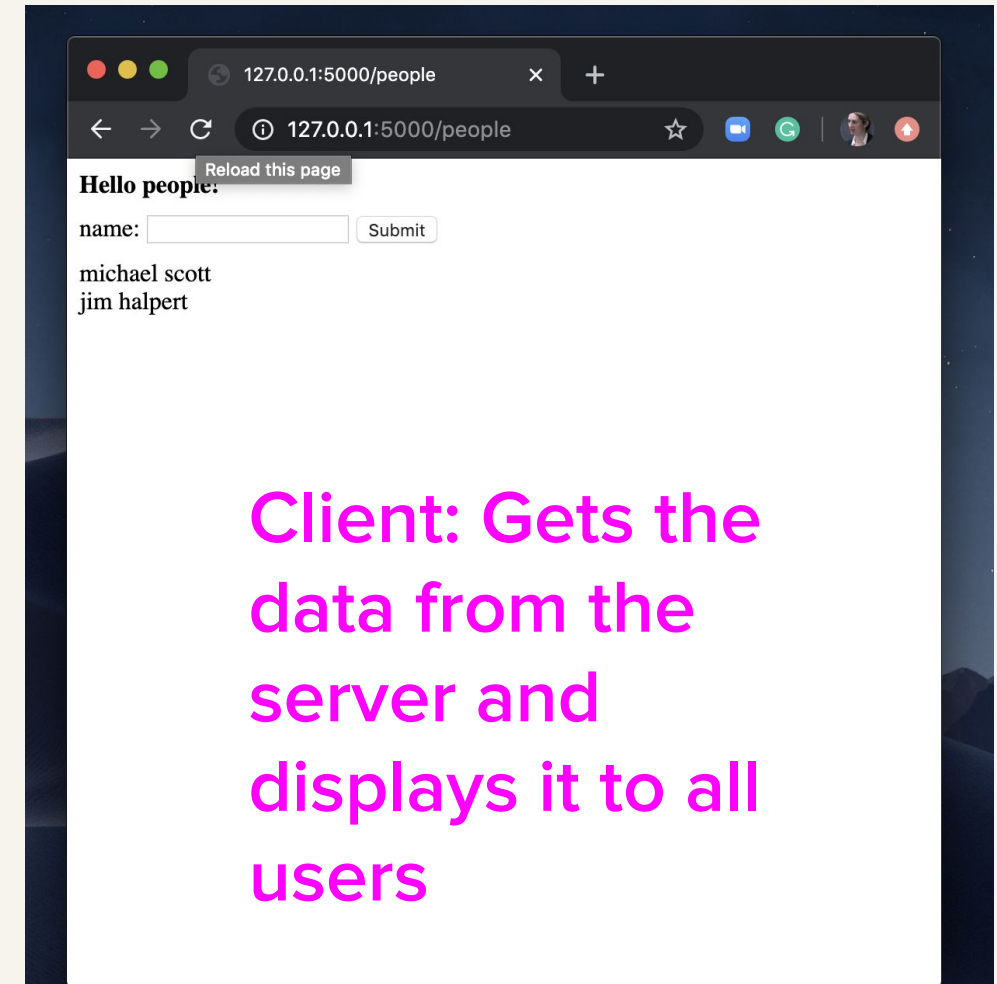


The image shows two windows from a macOS desktop. The top window is a terminal titled 'people — Python · Python server.py — 80x24'. It displays the output of running a Python server script. The logs show the server starting on http://127.0.0.1:5000/, handling several GET requests for /people, /static/people.js, and /favicon.ico, and a POST request to /add_name. The bottom window is a code editor titled 'server.py' showing the source code of the Flask application. The code imports Flask, render_template, Response, request, and jsonify. It initializes an app, sets a current_id, and defines a data list containing two entries: {'id': 1, 'name': 'michael scott'} and {'id': 2, 'name': 'jim halpert'}.

```
people — Python · Python server.py — 80x24
Last login: Sun Feb 16 09:18:51 on ttys001
Lydias-MacBook-Pro:people lydiachilton$ python server.py
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
* Restarting with stat
* Debugger is active!
* Debugger PIN: 723-907-492
127.0.0.1 - - [19/Feb/2020 07:07:16] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2020 07:07:17] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [19/Feb/2020 07:07:46] "GET /people HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2020 07:07:46] "GET /static/people.js HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2020 07:07:56] "POST /add_name HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2020 07:08:01] "GET /people HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2020 07:09:25] "POST /add_name HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2020 07:09:28] "GET /people HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2020 07:26:36] "GET /people HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2020 07:26:38] "GET /people HTTP/1.1" 200 -
^CLydias-MacBook-Pro:people lydiachilton$ python server.py
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
* Restarting with stat
* Debugger is active!
* Debugger PIN: 723-907-492
127.0.0.1 - - [19/Feb/2020 07:26:46] "GET /people HTTP/1.1" 200 -

server.py
1 from flask import Flask
2 from flask import render_template
3 from flask import Response, request, jsonify
4 app = Flask(__name__)
5
6
7 current_id = 2
8 data = [
9     {
10         "id": 1,
11         "name": "michael scott"
12     },
13     {
14         "id": 2,
15         "name": "jim halpert"
16     },
17 ]
```

Server:
Keeps the
data

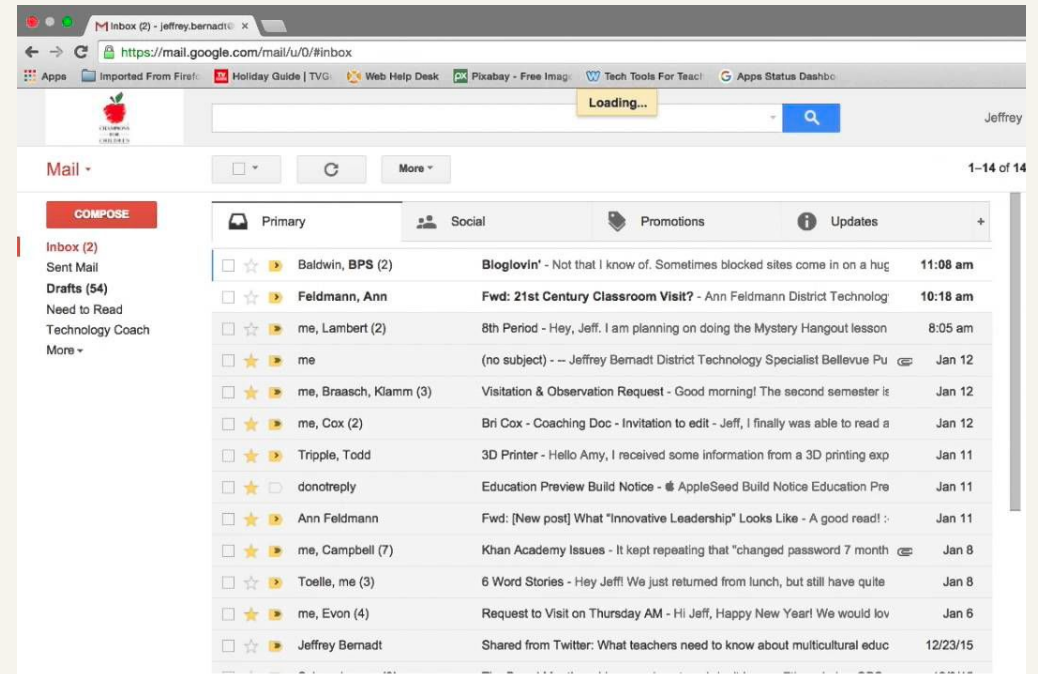


Client: Gets the
data from the
server and
displays it to all
users



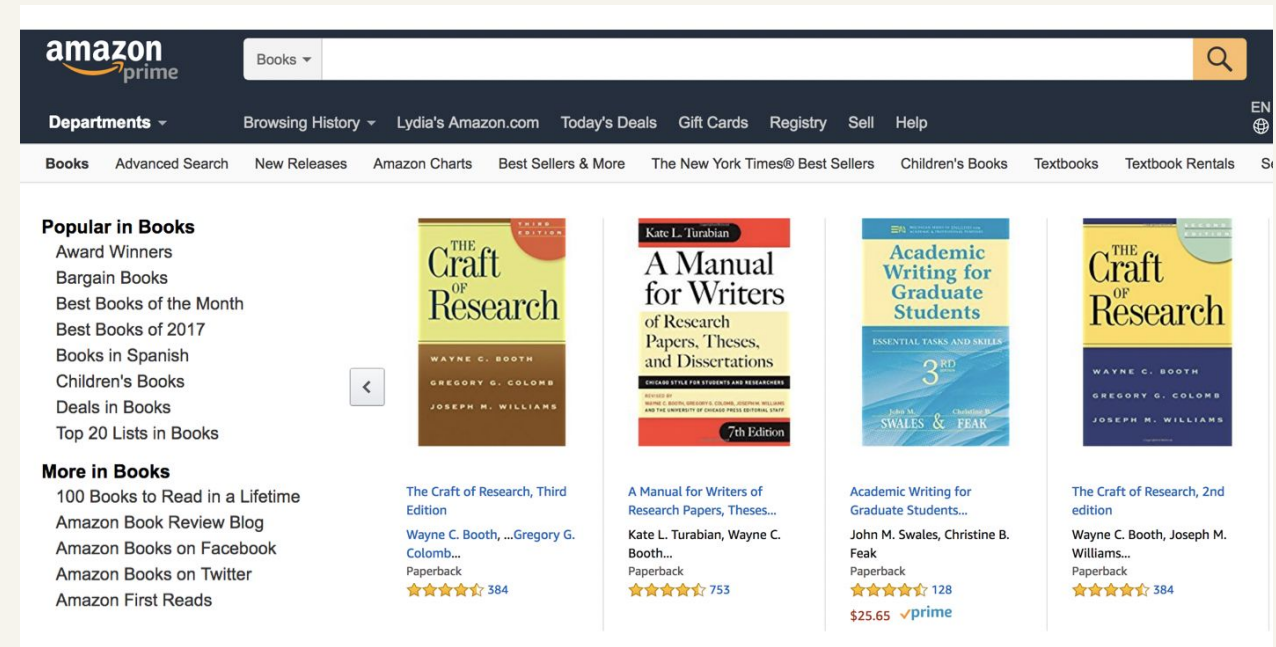
Gmail is a database of emails

```
emails = [  
  {  
    "from": "bollinger",  
    "to": "chilton",  
    "subject": "4170 is awesome!"  
  },  
  {  
    "from": "obama",  
    "to": "chilton",  
    "subject": "belated medal of freedom"  
  },  
]
```



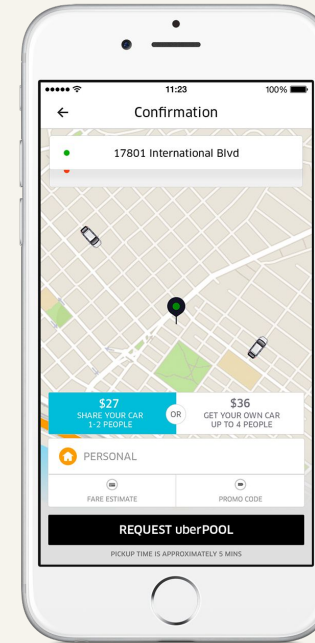
Amazon is a database of books

```
products = [  
  {  
    "title": "Ivy League Web Design",  
    "author": "chilton",  
    "stars": "5"  
  },  
  {  
    "title": "JavaScript and You",  
    "author": "chilton",  
    "stars": "6"  
  },  
]
```



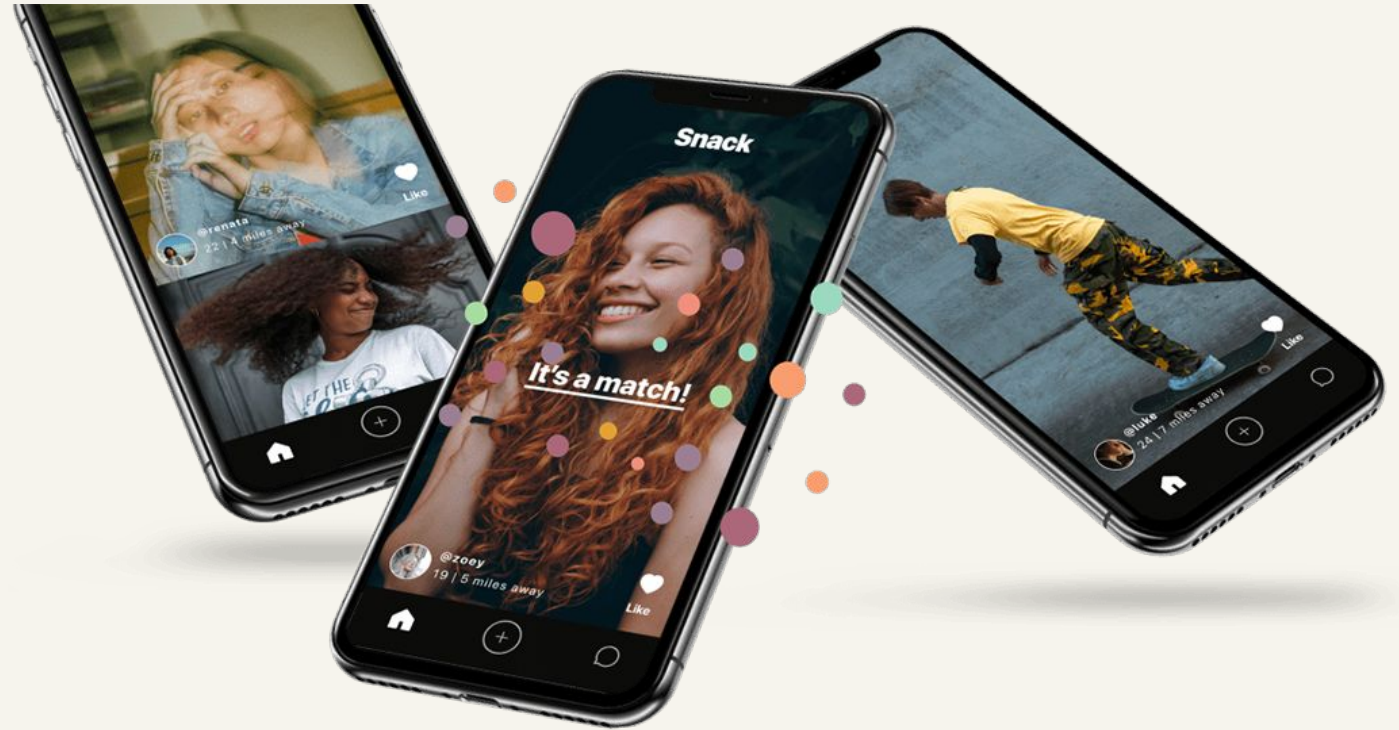
Uber is a database of **drivers**

```
cars = [  
  {  
    "location": "116 and broadway",  
    "driver": "kenny",  
    "car type": "uber XL"  
  },  
  {  
    "location": "times square",  
    "driver": "jen",  
    "car type": "normal"  
  },  
]
```



Snack is a database of **profiles**

```
profiles = [  
  {  
    "name": "maddy",  
    "image": "./maddy.png",  
    "likes": "1000",  
    "dislikes": 0,  
  },  
  {  
    "name": "julia",  
    "image": "./julia.png",  
    "likes": "1000",  
    "dislikes": 0,  
  },  
]
```



YouTube is a database of **videos**



windows 95 jennifer aniston matthew perry







9+





7:19 / 29:44



Windows 95 Video Guide with Matthew Perry & Jennifer Aniston

216,782 views

 2K  48  SHARE 



Favorite Videos 
Published on Dec 12, 2016

Watch all about the show Friends in a single playlist - <https://www.youtube.com/playlist?list...>
Subscribe and watch scenes from all seasons of Friends on my channel - <https://www.youtube.com/channel/UCA8V...>

SHOW MORE

SUBSCRIBE

201K

Up next

AUTOPLAY 



42:30

Friends - The One that Goes Behind the Scenes
Favorite Videos 
1.8M views



S11 E08
28:34

Would I Lie to You? - S11 E08 (NEW WILTY 19th January)
David Mitchell
Recommended for you
New



10:04

Actor Replacements That Totally Ruined The Show
Looper 
3M views



5:07

Inside Matthew Perry's Tragic Real Life Story
Nicki Swift 
952K views



42:29

Friends - ET On VH1: Behind The Friends Finale
Favorite Videos 
1M views



8:27

REACT S7 • E19 TEENS REACT TO WINDOWS 95
FBE 
12M views



4:53

Friends - Ross and Monica's Cousin
Favorite Videos 
3.6M views



Facebook is a database of posts

facebook

**PictureCorrect Photography Tips**
What is your Photography-Related New Year's Resolution??
15 minutes ago · Like · Comment

 Adel Gomaa and Richard Schneider like this.

**Sadi Molloy** get my business looking more professional: car signed: tick, making up portrait folders: tick, stickers & cd covers: still to do... its getting there though.
11 minutes ago · Flag

**PictureCorrect Photography Tips** My resolution for 2011 is to capture more landscape photos outside the U.S. and add more countries to my portfolio! What is yours?
-Richard
11 minutes ago · Flag

**James Donnelly** Larger concerts to cover and improve my portrait editing style/skill
10 minutes ago · Flag

**Jose Antonio Rubido** To push my photpography foward full throttle
9 minutes ago · Flag

**Deacon Jonathan Jerome Johnson** To learn more and enhance the quality of my shots!
5 minutes ago · Flag

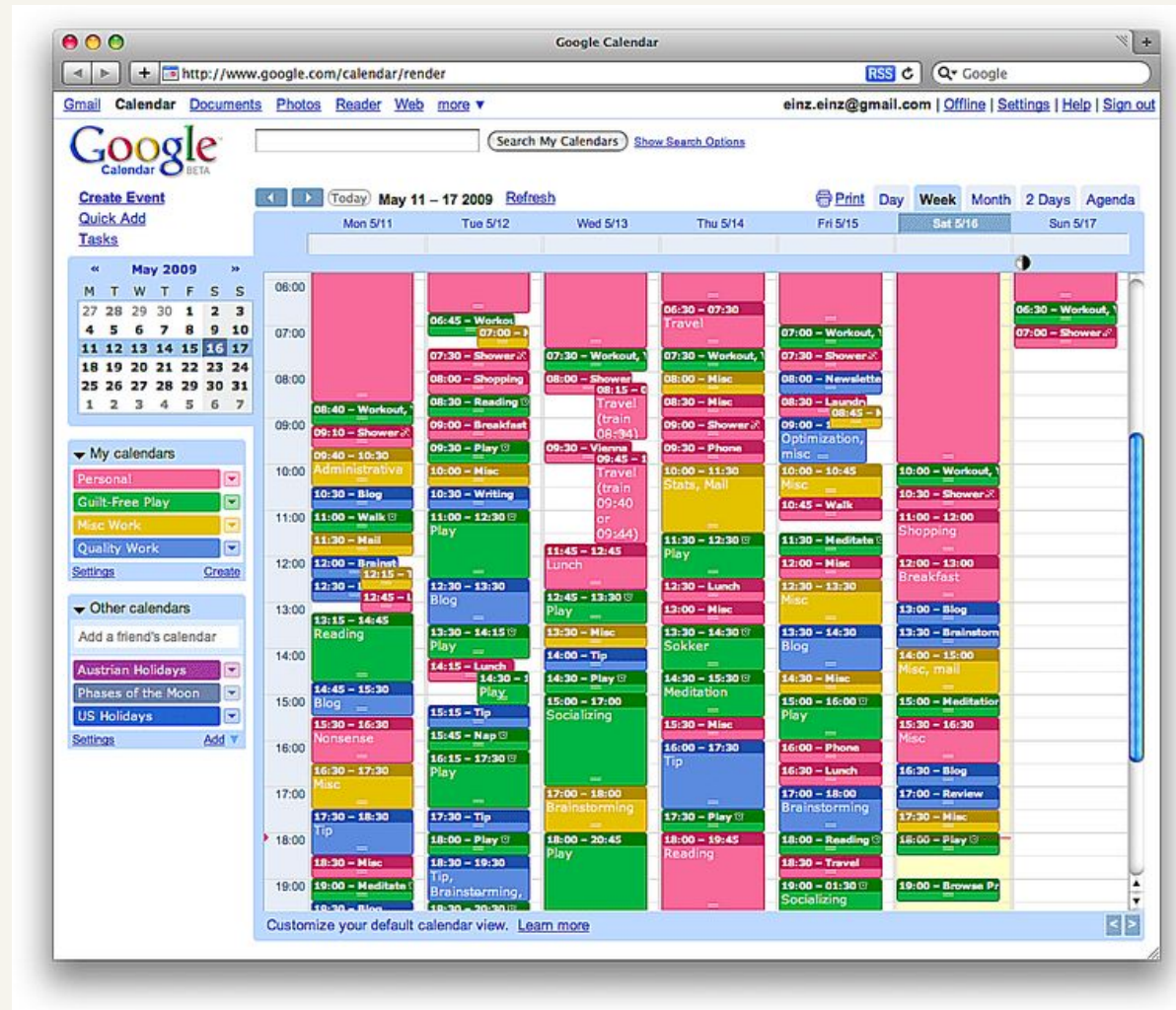
**Latu Peti Vimahi Manulua-hafoka** learn learn learn!!!
4 minutes ago · Flag

**Tina Taylor Hammer** Get my new business moving and have it pay for that lens and other equipment that I want (and a little for me too).
2 minutes ago · Flag

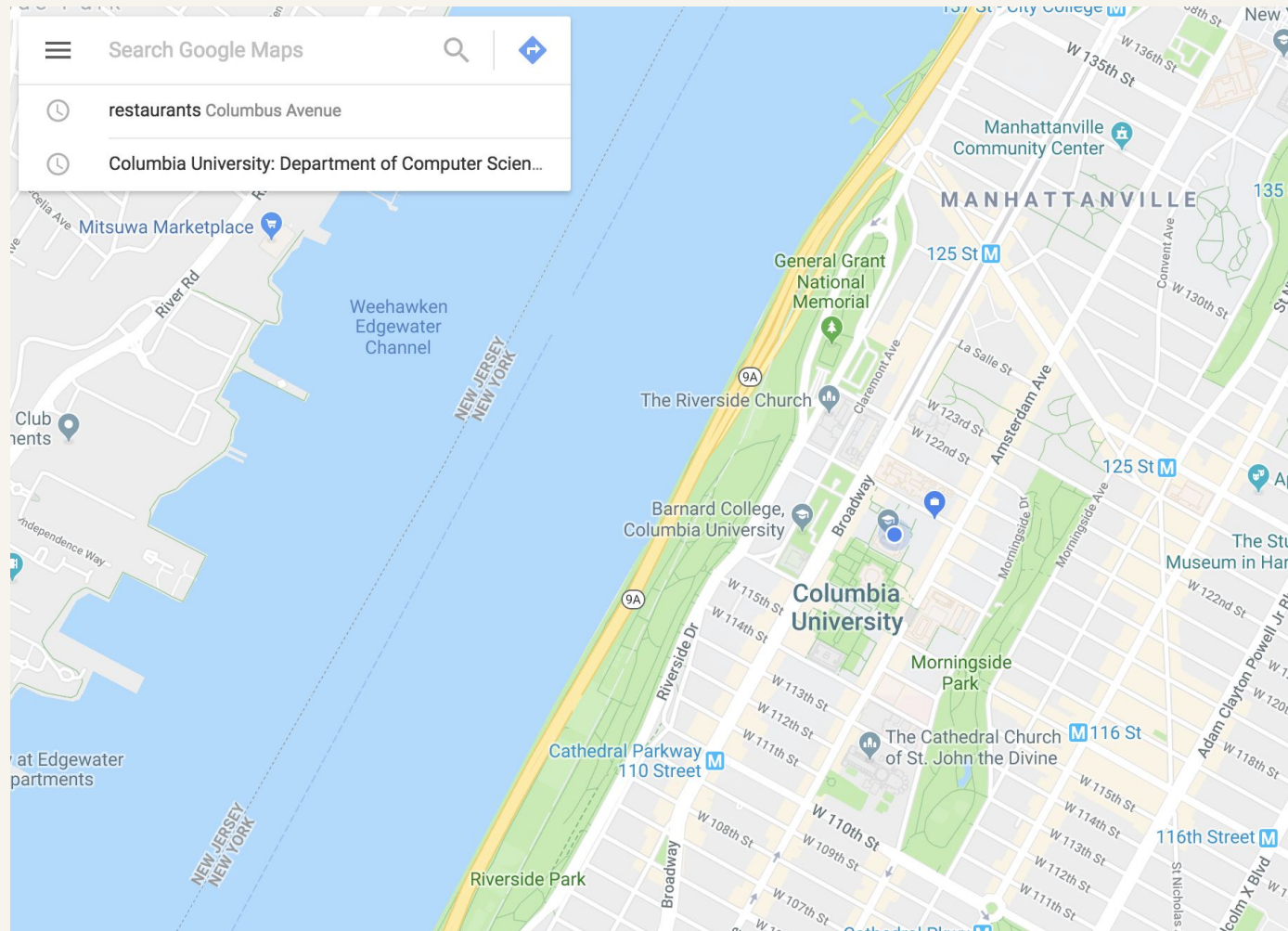
**Stefanie Bakewell** get a website up so that i can get clients full steam ahead
about a minute ago · Flag



Google Calendar is a database of **events**



Google Maps is a database of locations



New York Times is a database of news articles

SECTIONS

SEARCH

hmslydia

ENGLISH 中文 (CHINESE) ESPAÑOL

The New York Times

Friday, January 19, 2018 | Today's Paper | Video | 34°F | S. & P. 500 +0.19% ↑

WorldU.S. PoliticsN.Y. BusinessOpinionTechScienceHealthSportsArtsStyleFoodTravelMagazineT MagazineReal EstateALL

Trump in Shutdown Talks With Schumer

White House Invites Democratic Leader to Negotiate a Deal

By THE NEW YORK TIMES
49 minutes ago

- The government will shut down at 12:01 a.m. Saturday if lawmakers fail to get an agreement on some kind of spending bill.
- With Democrats balking, President Trump canceled plans to travel to his Florida resort and contacted Senator Chuck Schumer, the Democratic leader.
- **Timeline: How Congress Got to the Brink of a Shutdown**
- **Right and Left React to Prospect of Shutdown**



"We are not going to desert these young people," he said, referring to DACA beneficiaries.

Senator Bernie Sanders of Vt.

How the Debate in Congress Breaks Down

By ALICIA PARLAPIANO and WILSON ANDREWS
Lawmakers have until Friday at midnight to pass a short-term spending bill to avoid a government shutdown.

Senate to Vote With Shutdown at Stake

By THOMAS KAPLAN and SHERYL GAY STOLBERG 44 minutes ago
Senate Democrats appear ready to block legislation to keep the government funded past midnight, gambling that Mr. Trump will offer concessions on immigration and other issues.

574 Comments

The Chaos President vs. His Iron-Fisted Chief of Staff

By MAGGIE HABERMAN and JULIE

Opinion

EDITORIAL

Here's Another Fine Mess They've Gotten Us Into

Once again, Trump and Congress cannot perform a basic task, preventing a government shutdown.



- **Krugman: Doomsday Politics**
- **The Breakdown in Trust That Could Lead to a Shutdown**
- **Brooks: It's Time to Talk About the Power of Touch**
- **Goldberg: Is This the Collusion We Were Waiting For?**
- **Kristof: Swallowed by the Sea**
- **Leonhardt: A Warning for Democrats**
- **Egan: The Mad King**
- **The Shadow Safety Net**

FRANK BRUNI

Donald Trump's Radical Honesty

In his own wild way, this president is as truthful and transparent as they come.



Clubbable, but in the Worst Way

By MICHAEL GOLDFARB
Donald Trump is a man of his class — the nouveau-riche, country-club class.



- **How the Pro-Life Movement Has Promoted Liberal Values**
- **ICE Detained My Husband for Being an Activist**
- **Follow us on Twitter »**

TIMES INSIDER »

'I, Figure Skating Reporter': Full Circle With Tonya Harding

THE CROSSWORD »

Play Today's Puzzle



33m

The Justice Department wants to ask if you are a U.S. citizen in the 2020 census. But public distrust of the government could lead Hispanic people to avoid being counted.



5h

Can software predict crime? A study suggests that human amateurs are about as accurate as a popular software that many judges use to



VIDEO INTERVIEW





Craigslist is a database of random items for sale

CL new york > brooklyn > for sale > furniture - by owner

reply  prohibited  Posted 12 days ago [< prev](#) [next >](#)

★ **Gorgeous new 6ft. tall throne chair - \$1199 (Brooklyn)** 



condition: new

Gorgeous new 6ft. tall throne chair for \$1199obo. I will deliver it to you!

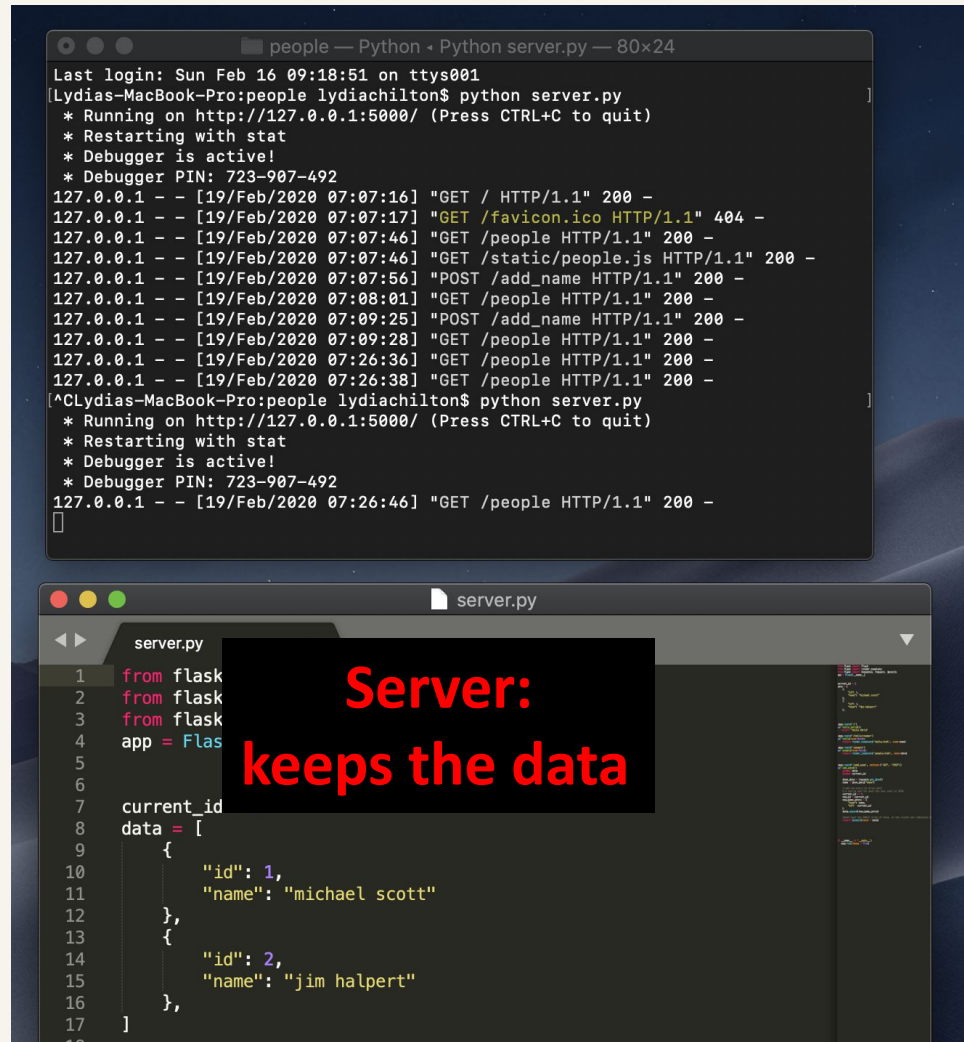
- do NOT contact me with unsolicited services or offers

post id: 6490476659 posted: 12 days ago updated: 34 minutes ago [email to friend](#) ♥ [best of](#) 

- [safety tips](#)
- [prohibited items](#)
- [product recalls](#)
- [avoiding scams](#)



We need to have another computer store and serve the data. That server is running a Python application called Flask.

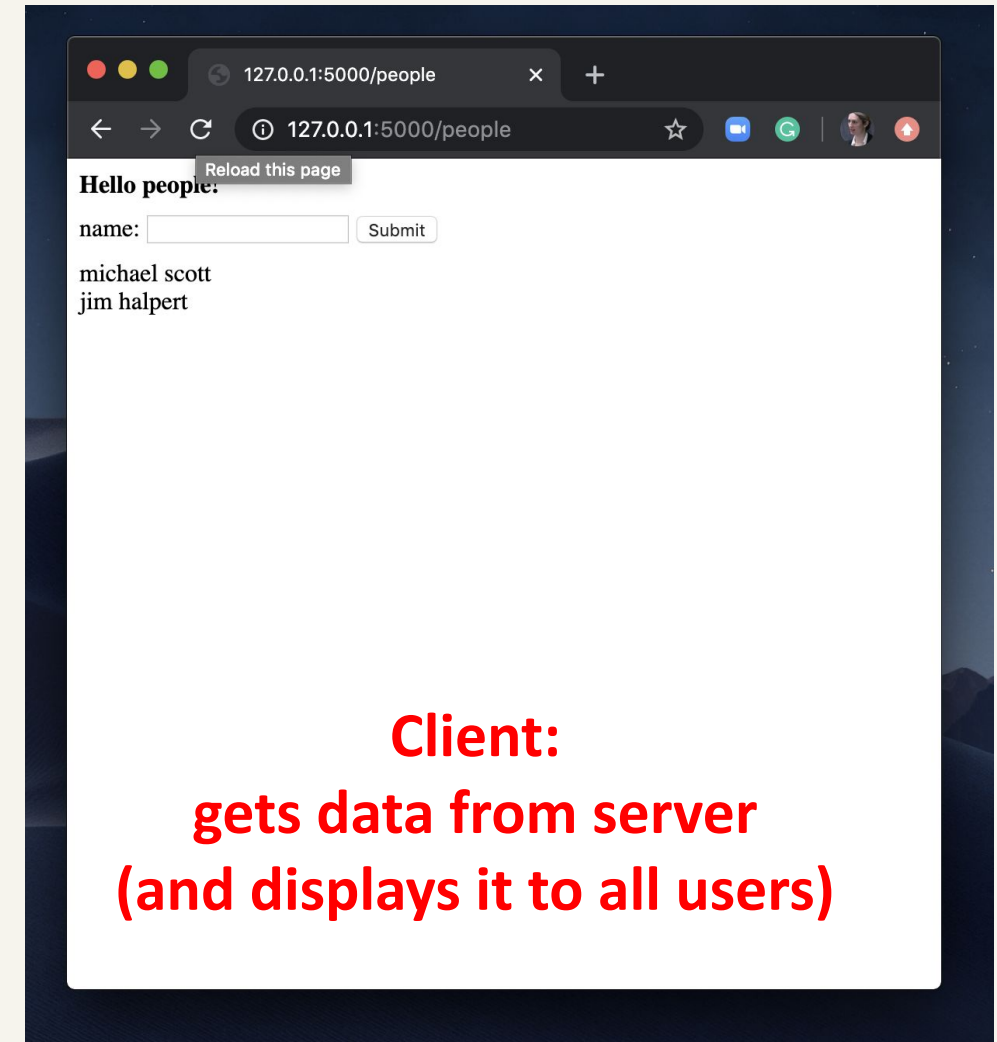


The image shows a terminal window and a code editor. The terminal window displays the output of running a Python server, including login information, server status, and a list of HTTP requests and responses. The code editor shows the Python code for the server, which uses Flask to create an application and store data in a list.

```
people — Python · Python server.py — 80x24
Last login: Sun Feb 16 09:18:51 on ttys001
Lydias-MacBook-Pro:people lydiachilton$ python server.py
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
* Restarting with stat
* Debugger is active!
* Debugger PIN: 723-907-492
127.0.0.1 - - [19/Feb/2020 07:07:16] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2020 07:07:17] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [19/Feb/2020 07:07:46] "GET /people HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2020 07:07:46] "GET /static/people.js HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2020 07:07:56] "POST /add_name HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2020 07:08:01] "GET /people HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2020 07:09:25] "POST /add_name HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2020 07:09:28] "GET /people HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2020 07:26:36] "GET /people HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2020 07:26:38] "GET /people HTTP/1.1" 200 -
^CLydias-MacBook-Pro:people lydiachilton$ python server.py
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
* Restarting with stat
* Debugger is active!
* Debugger PIN: 723-907-492
127.0.0.1 - - [19/Feb/2020 07:26:46] "GET /people HTTP/1.1" 200 -
[]

server.py
1 from flask
2 from flask
3 from flask
4 app = Flas
5
6
7 current_id
8 data = [
9     {
10         "id": 1,
11         "name": "michael scott"
12     },
13     {
14         "id": 2,
15         "name": "jim halpert"
16     },
17 ]
```

**Server:
keeps the data**



The image shows a web browser window displaying a simple web application. The browser's address bar shows the URL "127.0.0.1:5000/people". The page content includes a heading "Hello people:", a form with a label "name:" and a text input field, and a "Submit" button. Below the form, the names "michael scott" and "jim halpert" are displayed. A red overlay text is present on the right side of the image.

**Client:
gets data from server
(and displays it to all users)**



This website has no database. Why not?

User Interface Design

COMS 4170 · Spring 2018

[Home](#) [Syllabus](#) [Assignments ▾](#)

[Piazza](#)

Part 1 Build websites that suit the needs and abilities of users.

Part 2 When the needs and abilities of users are uncertain, design systems by learning from iteration and experimentation.

INSTRUCTOR

[Prof. Lydia Chilton](#)

OH: Tuesday 3-4 pm, CEPSR 612

Please contact staff through Piazza only

TAS

Tessa Hurr

Eleanor Murguia

Lucille Sui

WEEKLY SCHEDULE

Lecture

Mon, Wed 4:10–5:25pm, 413 Kent Hall

But most websites do have databases.

They allow you to both see and interact with data.

amazonprime

Books

Shop Valentine's Day Deals

Departments

Browsing History

Lydia's Amazon.com

Today's Deals

Gift Cards

Registry

Sell

Help

Books

Advanced Search

New Releases

Amazon Charts

Best Sellers & More

The New York Times® Best Sellers

Children's Books

Textbooks

Textbook Rentals

Sell Us Your Books

Best Books

Popular in Books

Award Winners

Bargain Books

Best Books of the Month

Best Books of 2017

Books in Spanish

Children's Books

Deals in Books

Top 20 Lists in Books

More in Books

100 Books to Read in a Lifetime

Amazon Book Review Blog

Amazon Books on Facebook

Amazon Books on Twitter

Amazon First Reads

Amazon Kindle

Kindle eBooks

Kindle Deals

Kindle Unlimited

Prime Reading

Spanish Kindle eBooks

Textbooks

Textbooks Store

Textbook Rentals

Sell Us Your Books

Kindle eTextbooks

The Craft of Research

A Manual for Writers of Research Papers, Theses, and Dissertations

Academic Writing for Graduate Students

The Craft of Research

A Manual for Writers of Research Papers, Theses, and Dissertations

facebook

PictureCorrect Photography Tips

What is your Photography-Related New Year's Resolution??

15 minutes ago · Like · Comment

Adel Gomaa and Richard Schneider like this.

Sadi Molloy get my business looking more professional: car signed: tick, making up portrait folders: tick, stickers & cd covers: still to do... its getting there though.

11 minutes ago · Flag

PictureCorrect Photography Tips My resolution for 2011 is to capture more landscape photos outside the U.S. and add more countries to my portfolio! What is yours?

Richard 11 minutes ago · Flag

James Donnelly Larger concerts to cover and improve my portrait editing style/skill

10 minutes ago · Flag

Jose Antonio Rubido To push my photpography foward full throttle

9 minutes ago · Flag

Deacon Jonathan Jerome Johnson To learn more and enhance the quality of my shots!

5 minutes ago · Flag

Latu Peti Vimahi Manulua-hafoka learn learn learn!!!

4 minutes ago · Flag

Tina Taylor Hammer Get my new business moving and have it pay for that lens and other equipment that I want (and a little for me too).

2 minutes ago · Flag

Stefanie Bakewell get a website up so that i can get clents full steam ahead

about a minute ago · Flag

Amazon Book Review

Weekend Reading

Resolution Cage Match

Graphic Novels for Kids

Sticky Fingers: Jann Wenner

See more posts

Mail

COMPOSE

Inbox (2)

Sent Mail

Drafts (54)

Need to Read

Technology Coach

More

Primary

Social

Promotions

Updates

Baldwin, BPS (2)

Feldmann, Ann

me, Lambert (2)

me

me, Braasch, Klamm (3)

me, Cox (2)

Tripple, Todd

donotreply

Ann Feldmann

me, Campbell (7)

Toelle, me (3)

me, Evon (4)

Jeffrey Bernadt

Bloglovin' - Not that I know of. Sometimes blocked sites come in on a hug

Fwd: 21

8th Peri

(no subj)

Visitation

Bri Cox

3D Print

Educatic

Fwd: [Ni

Khan Ac

6 Word

Request

Shared

craigslist

post to classifieds

my account

search craigslist

event calendar

help, faq, abuse, legal

avoid scams & fraud

personal safety tips

terms of use

privacy policy

system status

about craigslist

craigslist is hiring in sf

craigslist open source

craigslist blog

best-of-craigslist

craigslist TV

"craigslist joe"

craig connects

progressive directory

weather quake tide

community

local news

lost+found

musicians

childcare

classes

events

politics

rideshare

volunteers

personals

strictly platonic

women seek women

women seeking women

men seeking men

misc romance

casual encounters

missed connections

rants and raves

discussion forums

antiques

appliances

arts+crafts

atv/utv/sno

auto parts

heavy equip

household

jewelry

materials

motorcycles

boats

books

business

cars+trucks

cds/dvd/vhs

cell phones

clothes+acc

collectibles

computers

electronics

farm+garden

free

furniture

garage sale

general

heavy equip

household

jewelry

materials

motorcycles

music instr

photo+video

rvs+camp

sporting

tools

toys+games

trailers

video gaming

wanted

services

automotive

legal

lessons

marine

pet

real estate

for sale

real estate

rental / wholesale

sales / biz dev

salon / spa / fitness

security

skilled trade / craft

software / qa / dba

systems / network

technical support

transport

tv / film / video

web / info design

writing / editing

[ETC]

[part-time]

gigs

computer

creative

labor

crew

talent

domestic

writing

english

nearby ci

bakersfield

chico

fresno

gold country

hanford

humboldt

inland empire

kiamath falls

las vegas

los angeles

medford

mendocino co

merced

modesto

monterey

orange co

palm springs

redding

reno

roseburg

sacramento

san luis obispo

santa barbara

santa maria

siskiyou co

stockton

susville

ventura

visalia-lulare

yuba-sutter

us states

us cities

canada

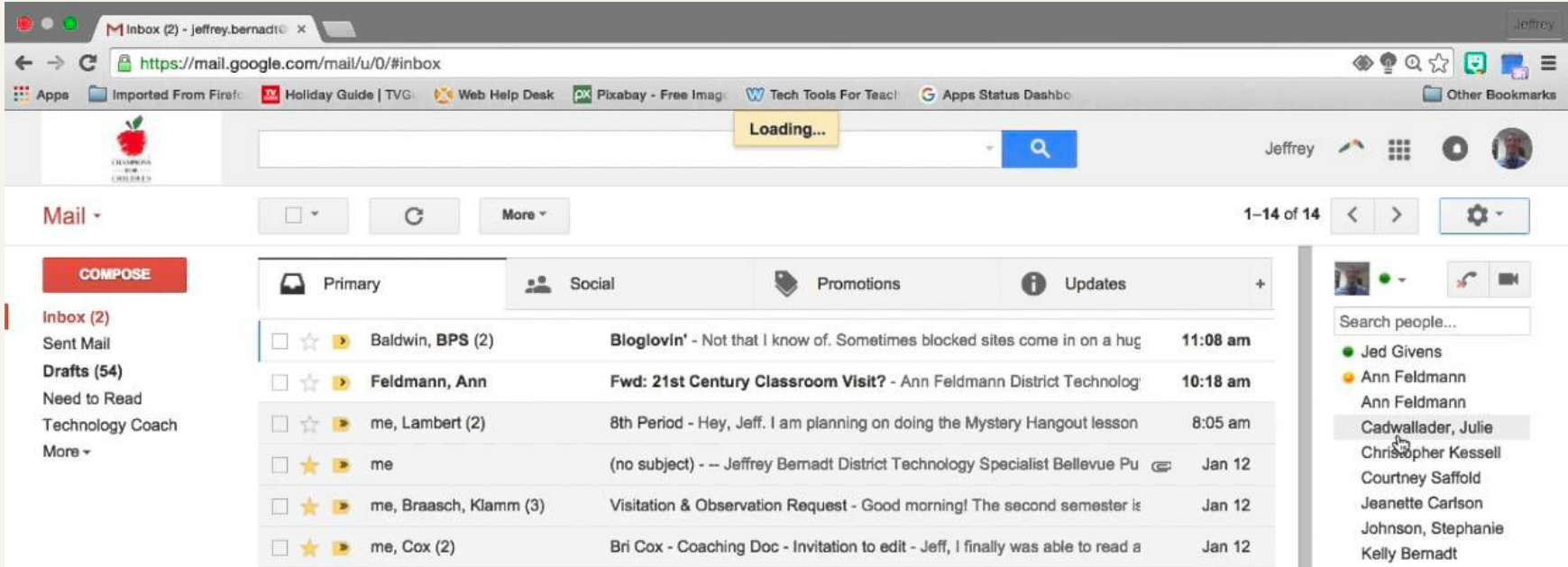
ci worldwide

COMS 4170 · SPRING 2026

Interacting with Data



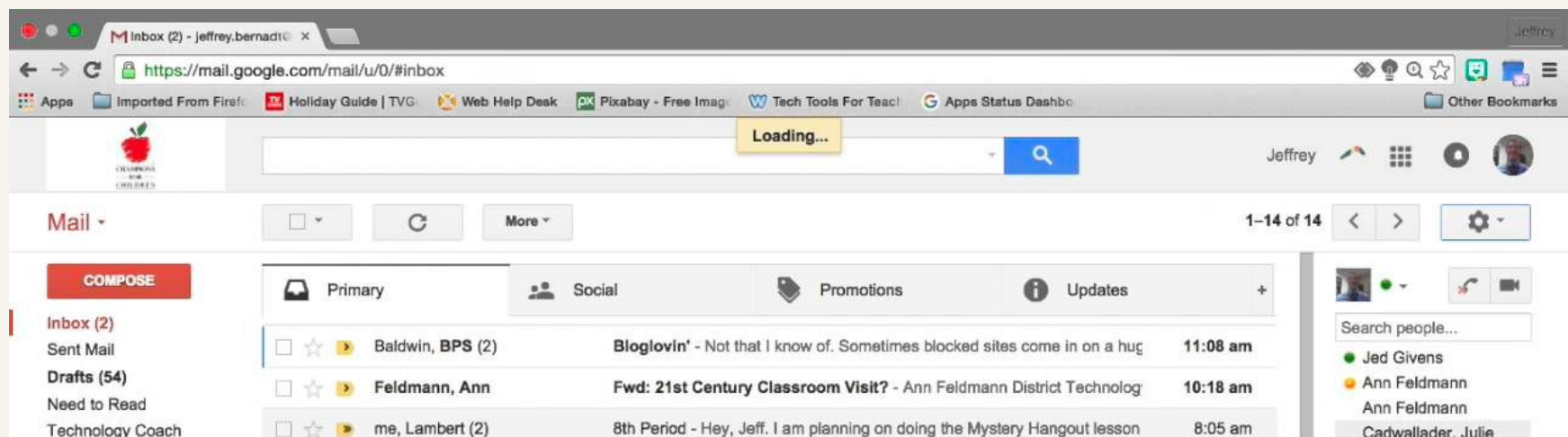
Gmail is a database of emails



ID	From	To	Subject	Body	Time
1	Baldwin	Jeffrey	Blogloving'	Not that I know of...	158904993585835
2	Ann Feldman	Jeffrey	Fwd: Classroom visit?	Ann Feldman District Technology	158904993585923
...					

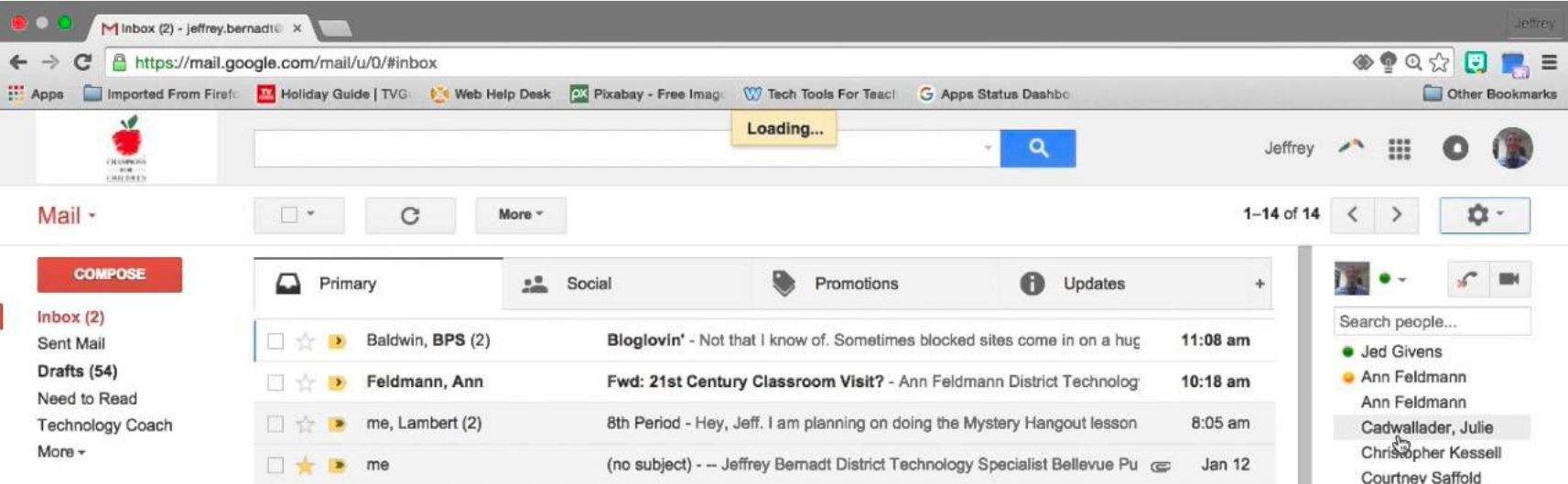
All emails are stored in one big database.

How do we only show Jeffrey's email to Jeffrey?



ID	From	To	Subject	Body	Time
1	Baldwin	Jeffrey	Blogloving'	Not that I know of...	158904993585835
2	Ann Feldman	Jeffrey	Fwd: Classroom visit?	Ann Feldman District Technology	158904993585923
3	Lydia Chilton	Lee Bollinger	Bowling tuesday	Hey dude! Donnie and I were thinking of going bowling...	158904993443772
...					

Four basic functions of persistent storage



ID	From	To	Subject	Body	Time
1	Baldwin	Jeffrey	Blogloving'	Not that I know of...	158904993585835
2	Ann Feldman	Jeffrey	Fwd: Classroom visit?	Ann Feldman District Technology	158904993585923
3	Lydia Chilton	Lee Bollinger	Bowling tuesday	Hey dude! Donnie and I were thinking of going bowling...	158904993443772

Four basic functions of persistent storage

	Method	Path	Server-side action
CREATE	POST	/movies	Create a movie with provided data
READ	GET	/movies	None
UPDATE	PUT	/movies/<int:id>	Update the movie with the provided data
DELETE	DELETE	/movies/<int:id>	Delete a single movie

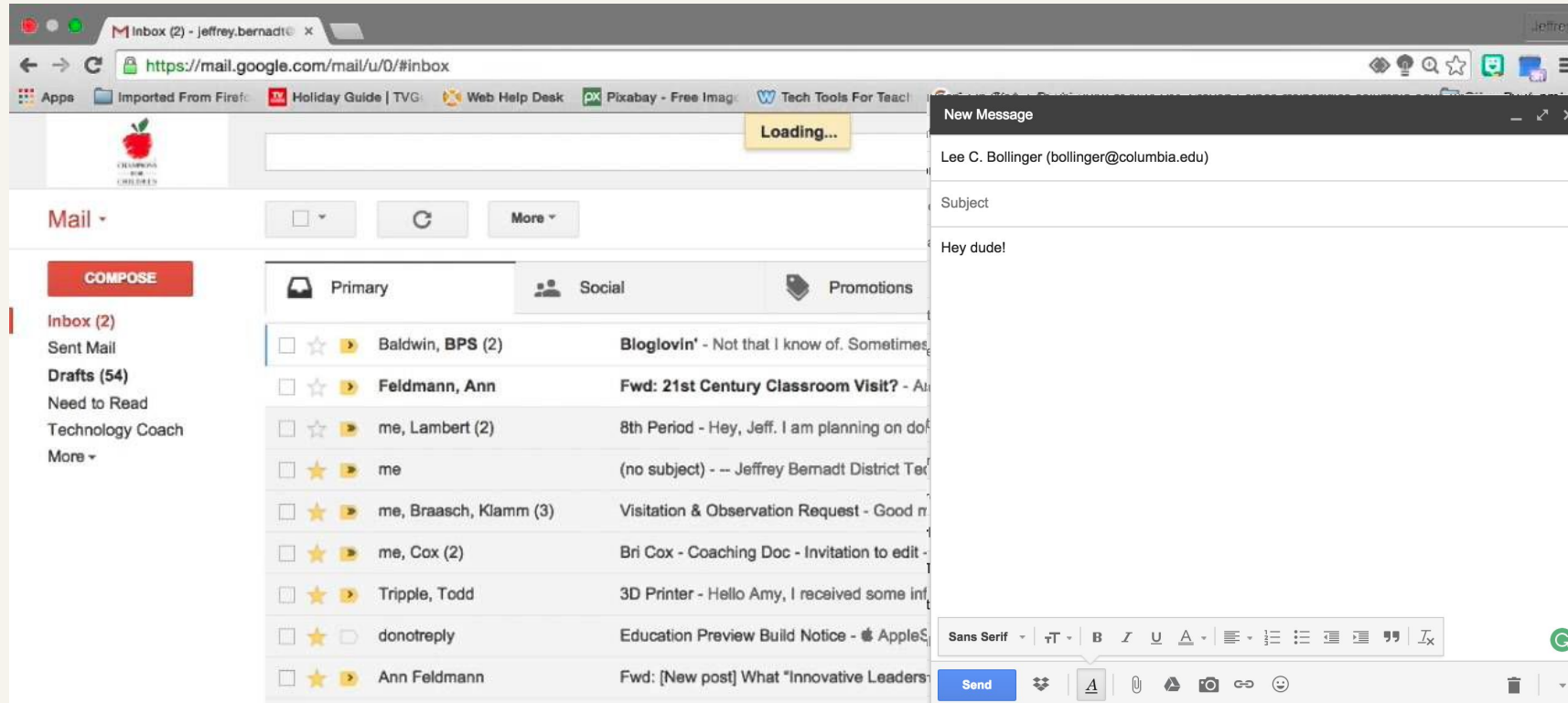


Four basic functions of persistent storage

	Method	Path	Server-side action
CREATE	POST	/movies	Create a movie with provided data
READ	GET	/movies	None



CREATE: How do users create data in GMail?

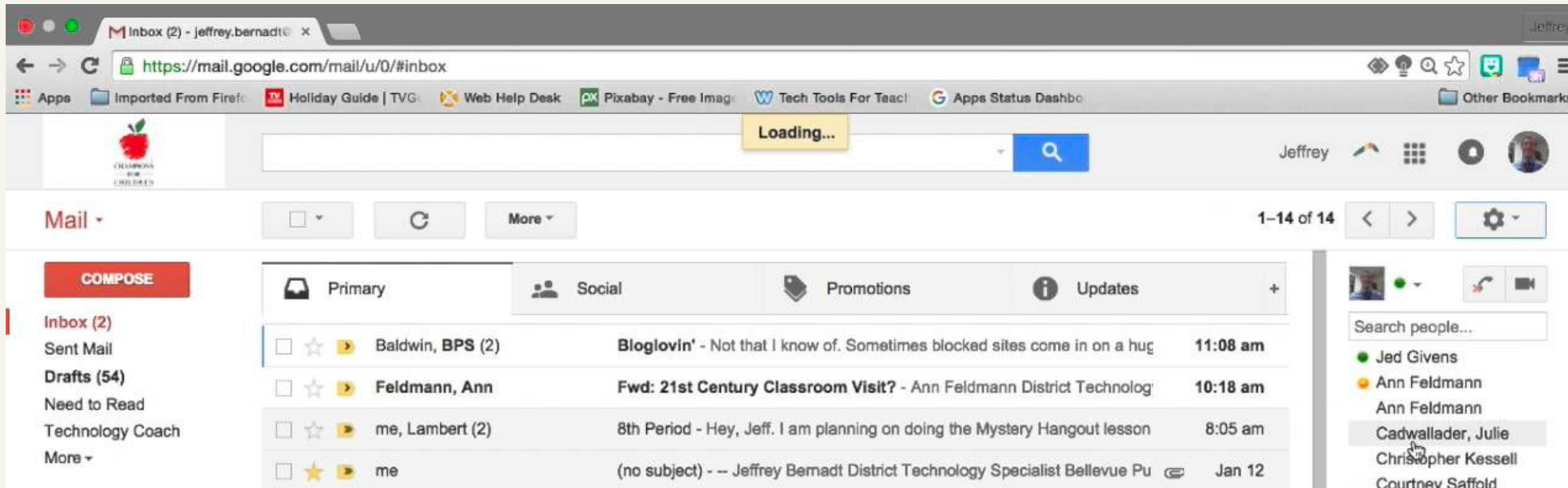


1. Compose a new email and send it
2. Reply to an email



READ: How do users read data in Gmail?

How do they see different portions of the database?

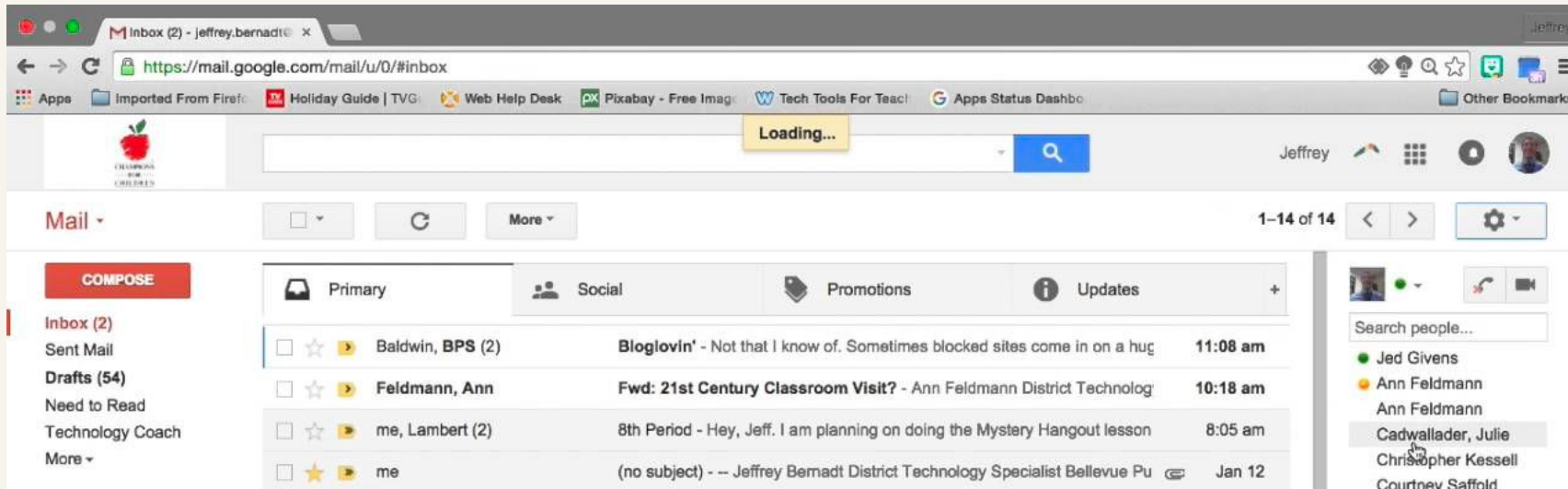


1. Load the page
2. Search the inbox



UPDATE: How do users update data in GMail?

What do they update? And how do they do it in the interface?

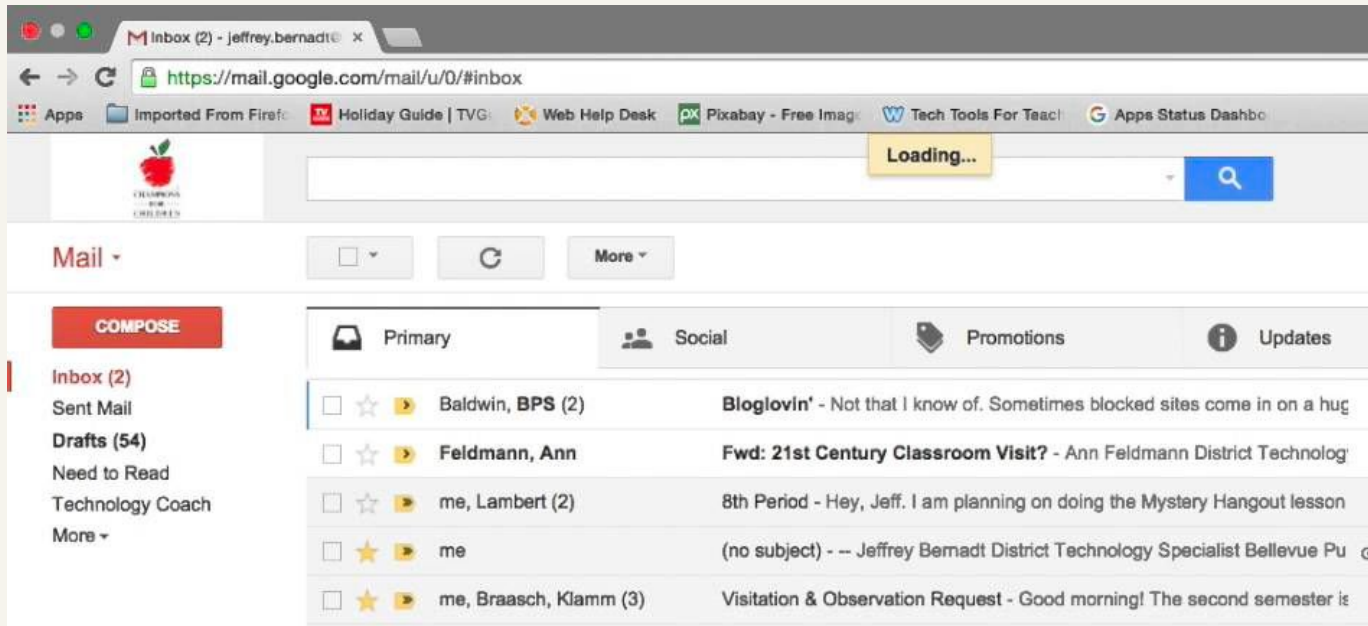


1. Read an email (it gets marked as “read”)
2. Star an email
3. Add or remove a label
4. Reply to an email (the original email gets updated to point to the reply)



DELETE: How do users delete data in GMail?

First, what objects can be deleted in GMail?



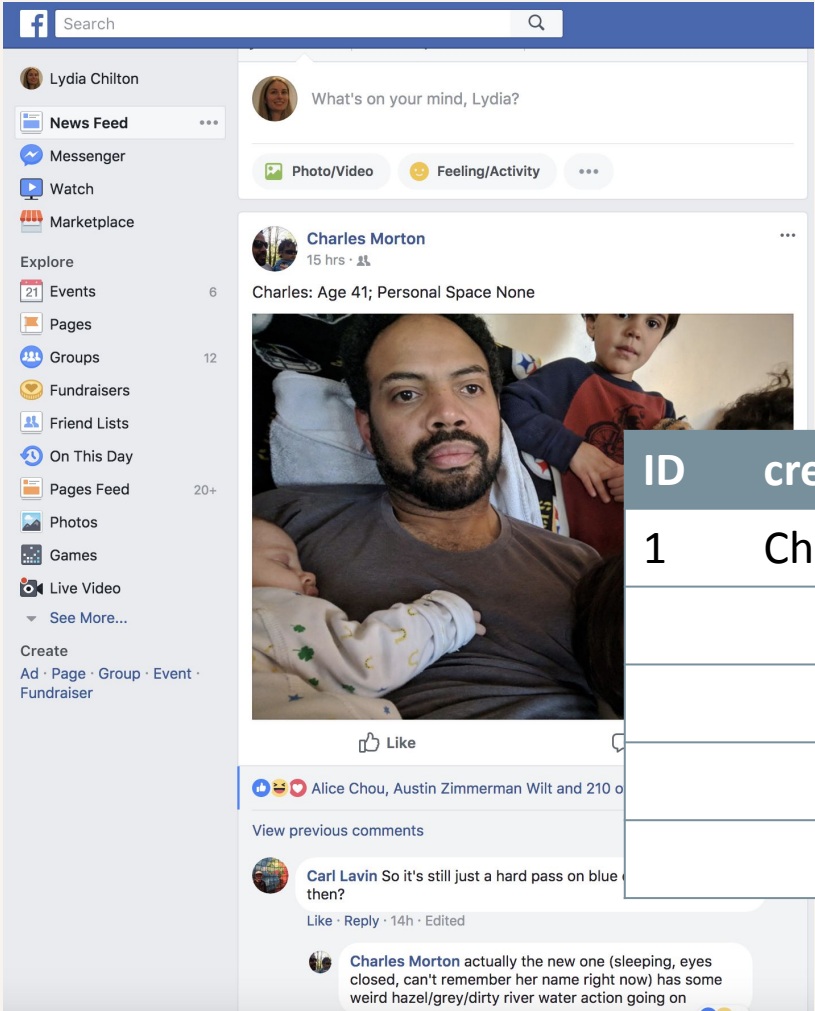
Deleting an email doesn't actually delete it – other users may still have it.

But it does UPDATE the email and mark it as deleted, so it doesn't show up for you.

1. Discard a draft
2. Leave spam untouched for 30 days



Facebook is a database of posts

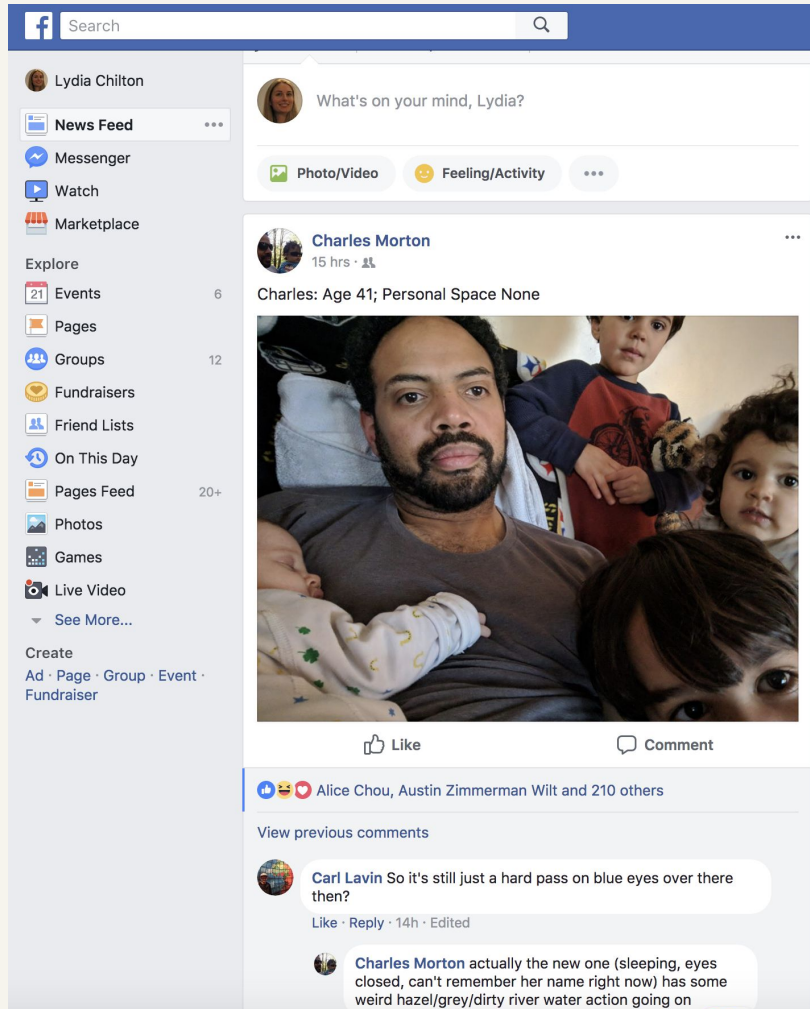


ID	creator	text	Likes	Time
1	Charles Morton	Charles Age 41: Personal..	210	15890499

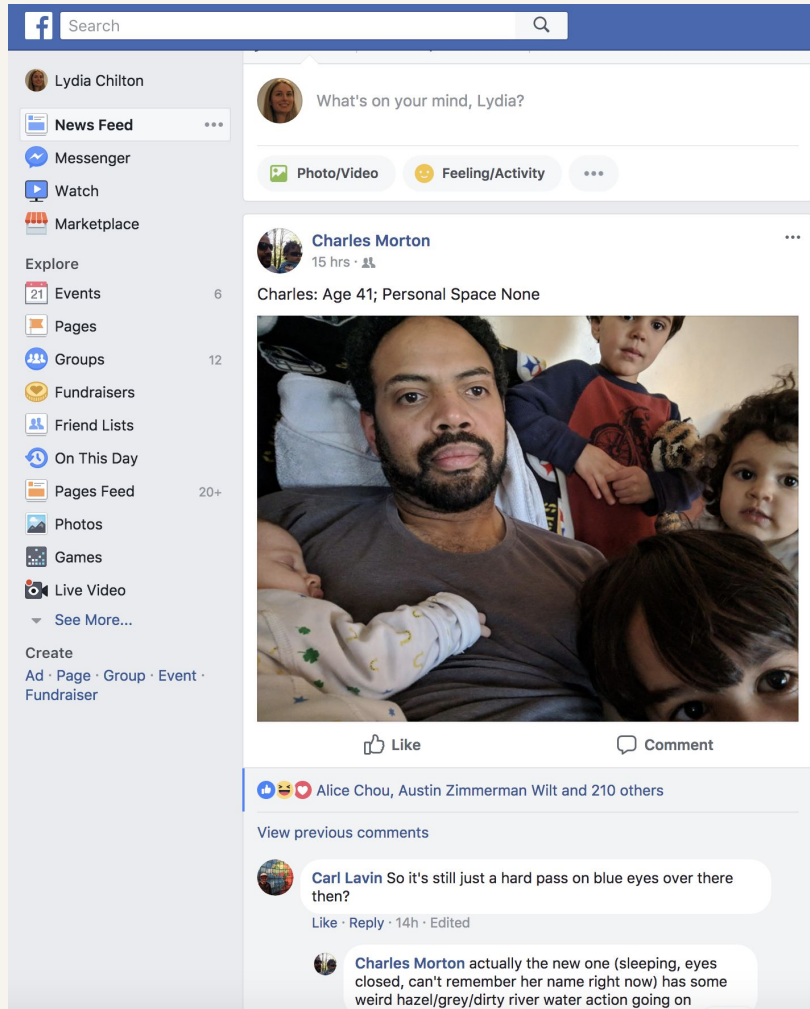


CREATE: How do users create data in Facebook?

1. Create Post
2. Create reply/response



READ: How do users read data in Facebook?



1. Load the page
2. Search for posts
3. Scroll – and it will autoload posts



UPDATE: How do users update data in Facebook?

1. Like



DELETE: How do users delete data in Facebook?

1. Delete the post?



The screenshot displays the Google Calendar web application. At the top, the browser address bar shows the URL <http://www.google.com/calendar/render>. The navigation bar includes links for Gmail, Calendar, Documents, Photos, Reader, Web, and more. The user's email address, einzel.einzel@gmail.com, and links for Offline, Settings, Help, and Sign out are also present.

The main interface features a search bar, a "Search My Calendars" button, and a "Show Search Options" link. Below this, the "Create Event" link is highlighted. The calendar view is set to "Today" (May 11, 2009) and is displayed in a weekly format. The days of the week are listed at the top: Mon 5/11, Tue 5/12, Wed 5/13, Thu 5/14, Fri 5/15, Sat 5/16, and Sun 5/17.

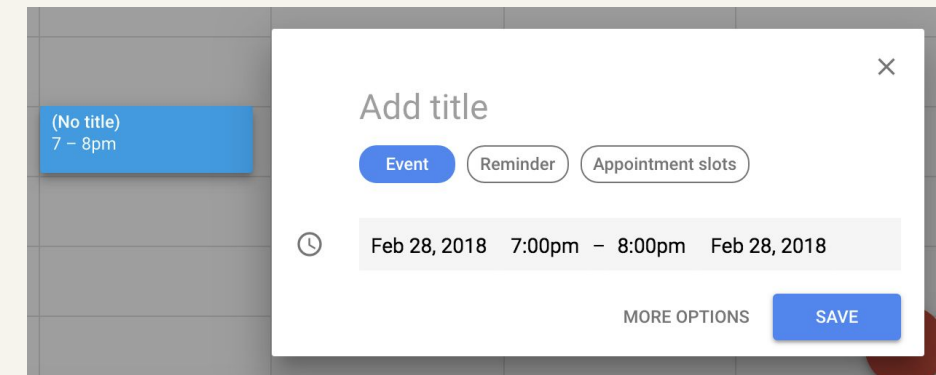
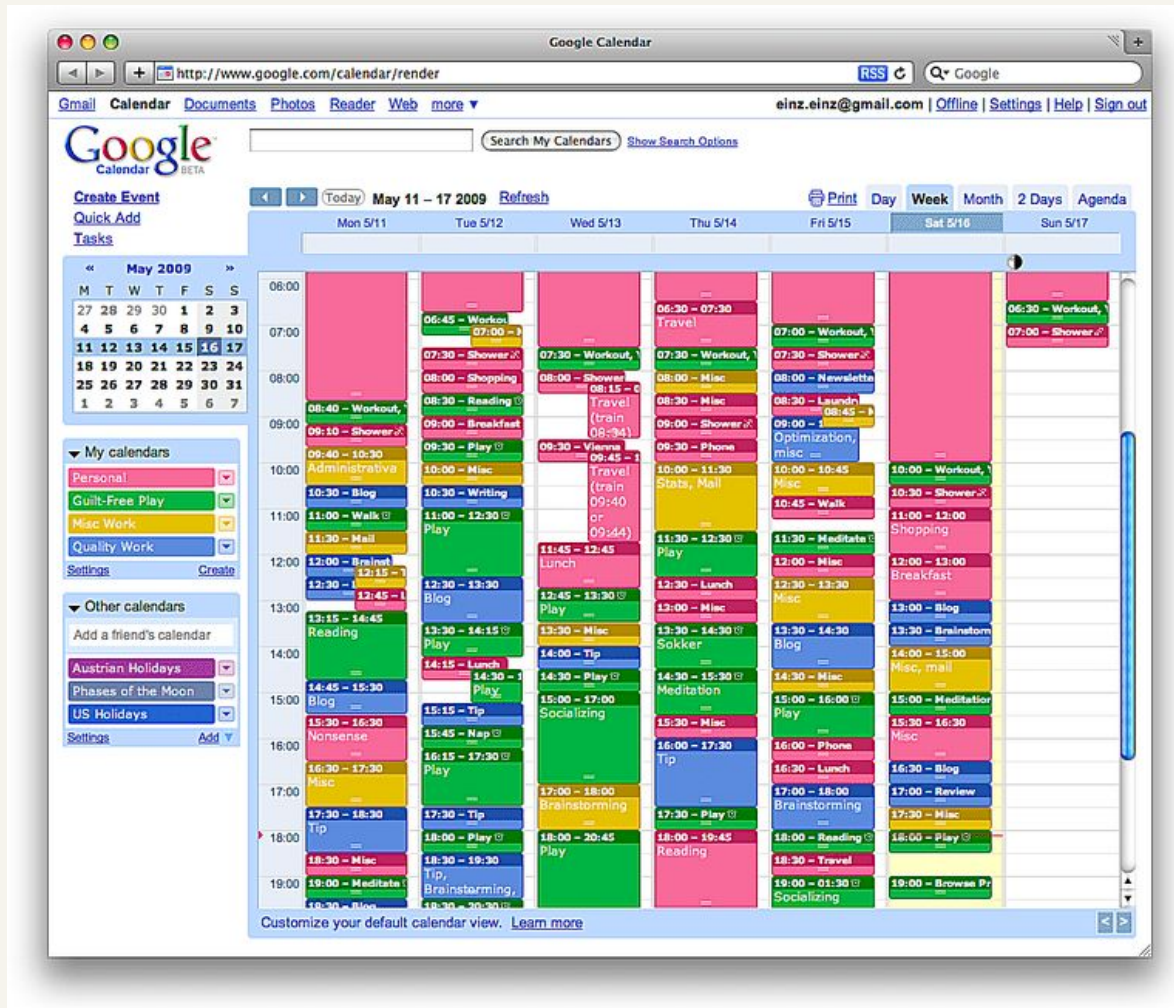
The left sidebar contains several sections:

- My calendars:** A list of calendars including Personal, Guilt-Free Play, Misc Work, and Quality Work, each with a color-coded icon and a dropdown arrow.
- Other calendars:** A section for adding a friend's calendar, with examples like Austrian Holidays, Phases of the Moon, and US Holidays.

The main calendar area shows a detailed hourly schedule. Events are color-coded to match the calendars they belong to. For example, on Monday (May 11), there are events like "06:45 - Workout", "07:30 - Shower", "08:00 - Shopping", "08:30 - Reading", "09:10 - Shower", "09:40 - Administrative", "10:30 - Blog", "11:00 - Walk", "11:30 - Mail", "12:00 - Breakfast", "12:30 - Lunch", "13:15 - Reading", "14:45 - Blog", "15:30 - Nonsense", "16:30 - Misc", "17:30 - Tip", "18:30 - Misc", and "19:00 - Meditation".

Overlaid on the right side of the screenshot is a portion of a document titled "Creator Statement". The visible text includes the name "Chilton" and the month "Feb", which appears to be part of a larger table or list.

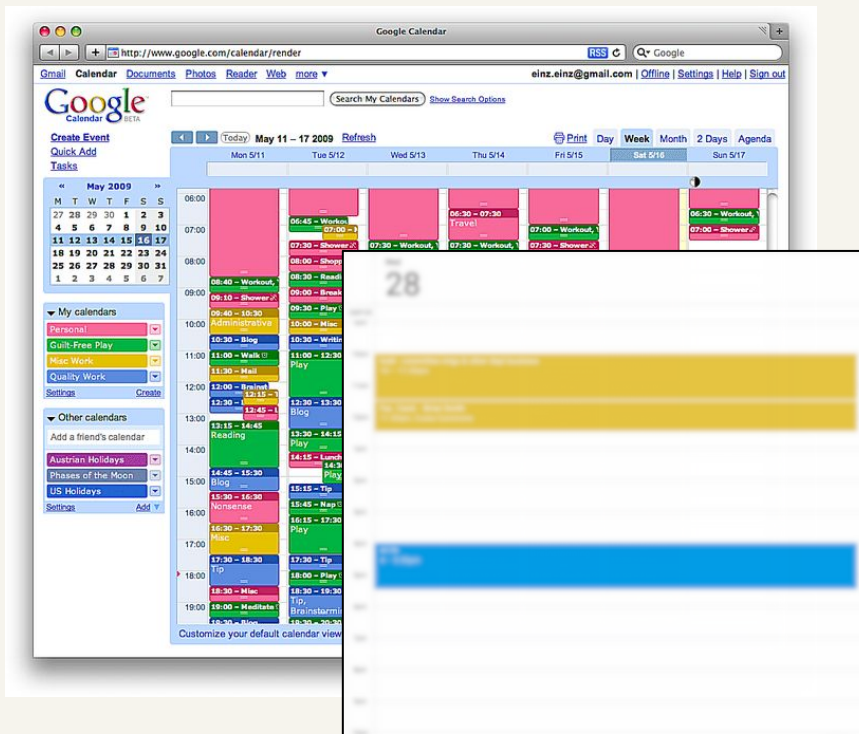
CREATE: How do users create data in Calendar?



READ: How do users read data in Calendar?

How do they see different portions of the database?

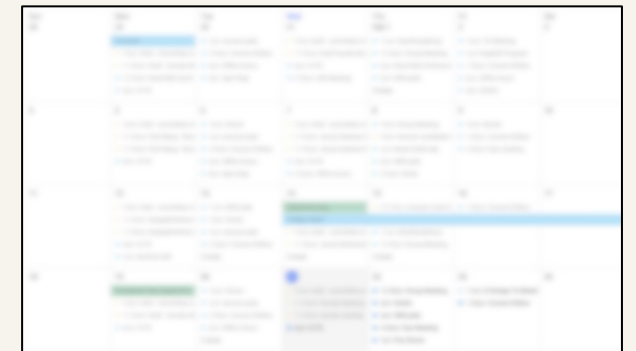
1. Load the page
2. Choose different views



Day view



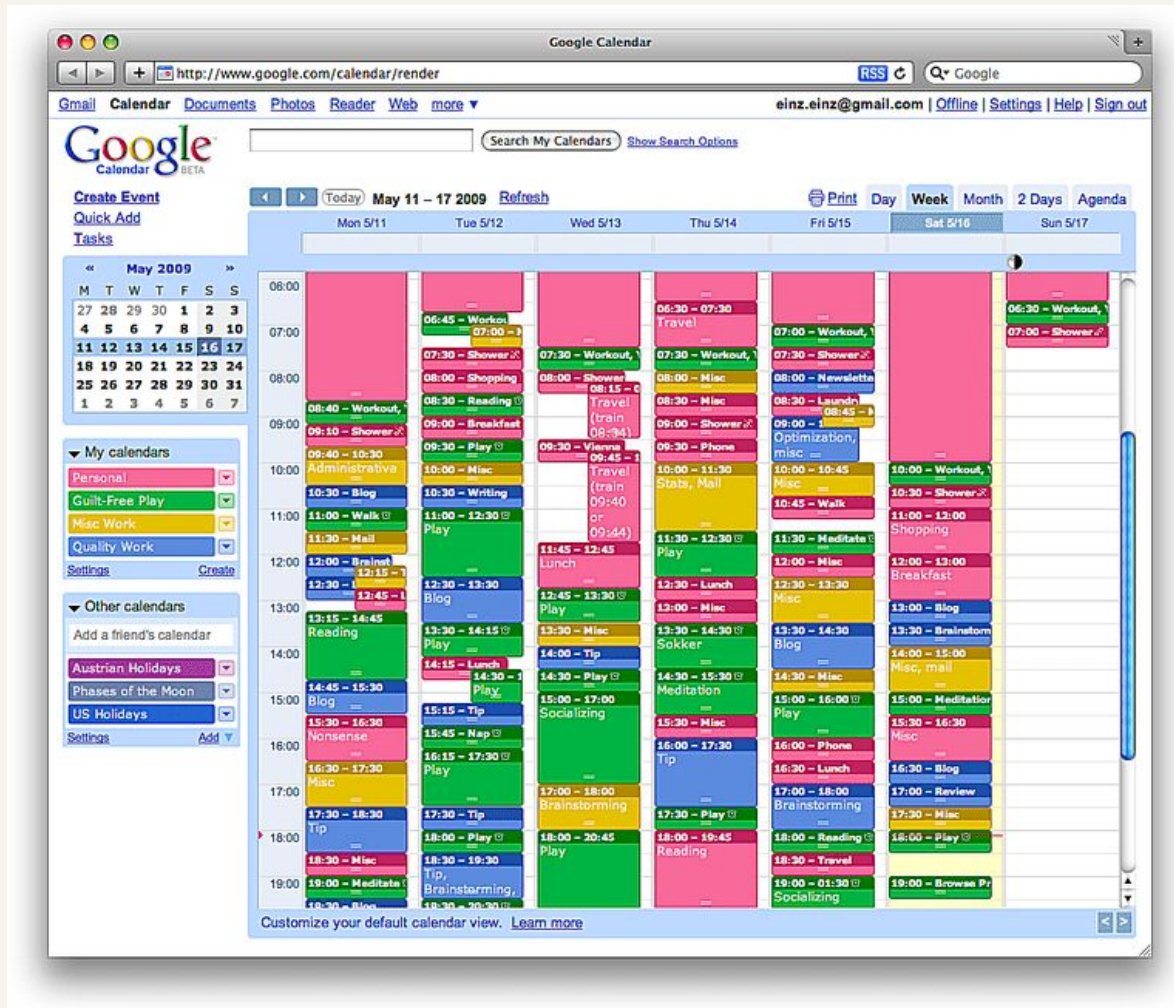
Week view



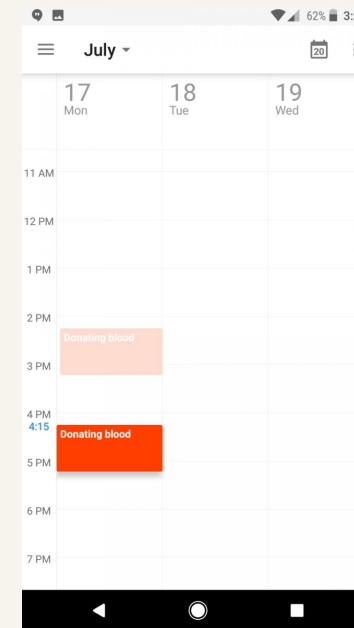
Month view



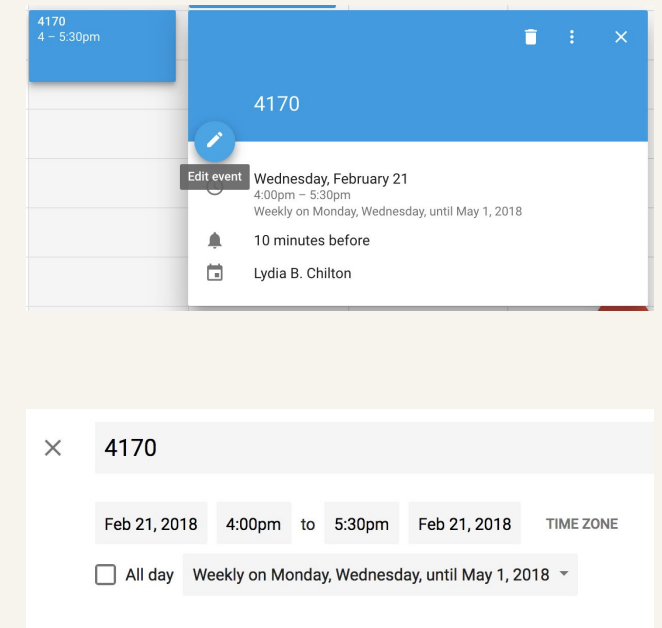
UPDATE: How do users update data in Calendar?



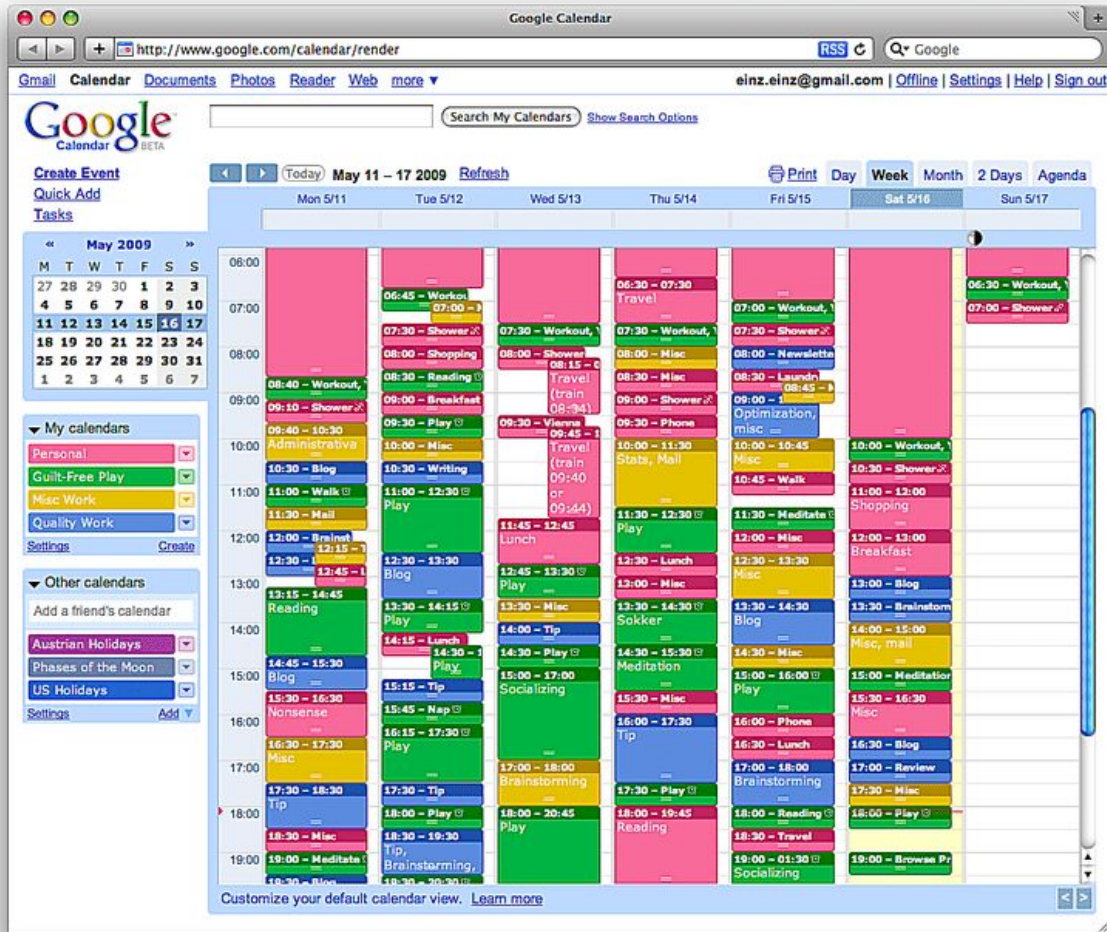
Drag and drop



Select an event and edit a form



DELETE: How do users delete data in Calendar?



Selecting an event and clicking delete



CRUD: Operations for interacting with a database

Create

New Message

To Lee C. Bollinger (bollinger@columbia.edu) ✕ |

From Lydia Chilton <chilton@cs.columbia.edu> ▾

Subject

Hey dude!

 What's on your mind, Lydia?

(No title)
7 - 8pm

Add title

Event Reminder Appointment slots

🕒 Feb 28, 2018 7:00pm - 8:00pm Feb 28, 2018

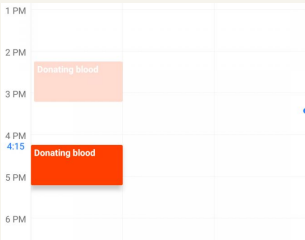
MORE OPTIONS SAVE

Read

Baldwin, BPS (2)	Bloglovin' - Not that I know of. Sometimes bl
Feldmann, Ann	Fwd: 21st Century Classroom Visit? - Ann
me, Lambert (2)	8th Period - Hey, Jeff. I am planning on doing
me	(no subject) - -- Jeffrey Bernadt District Tchr



Update



Delete



Delete Post?

Are you sure you want to delete it?

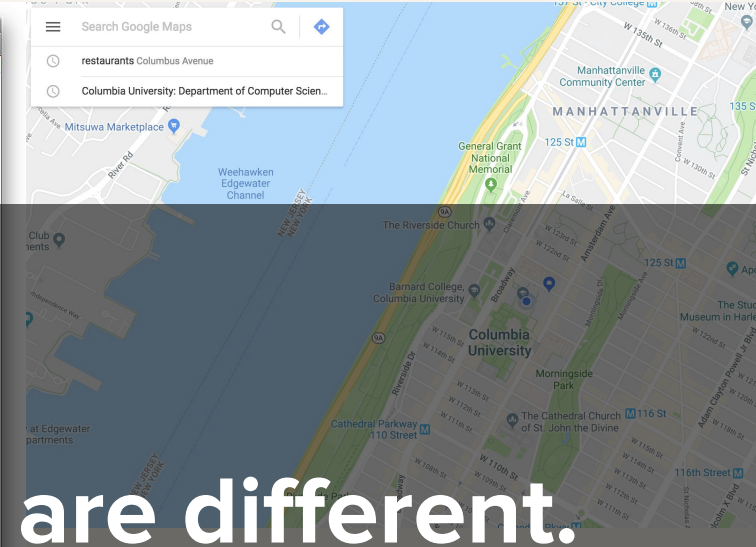
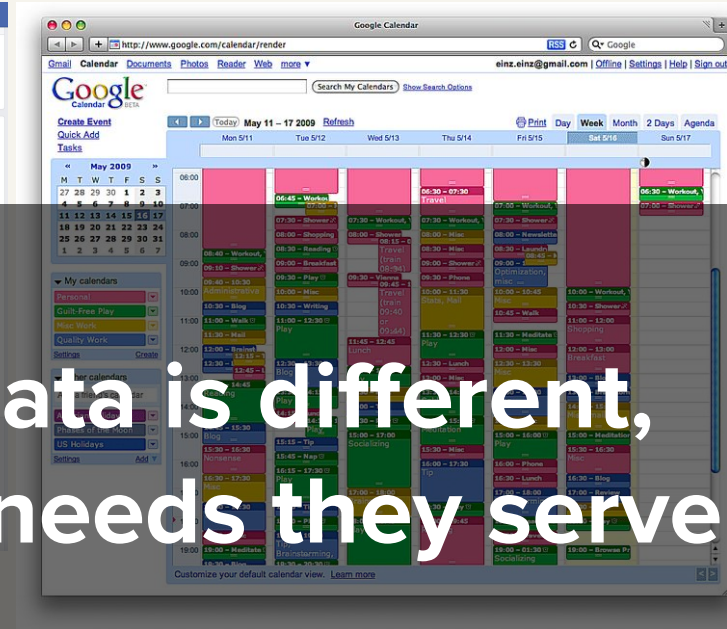
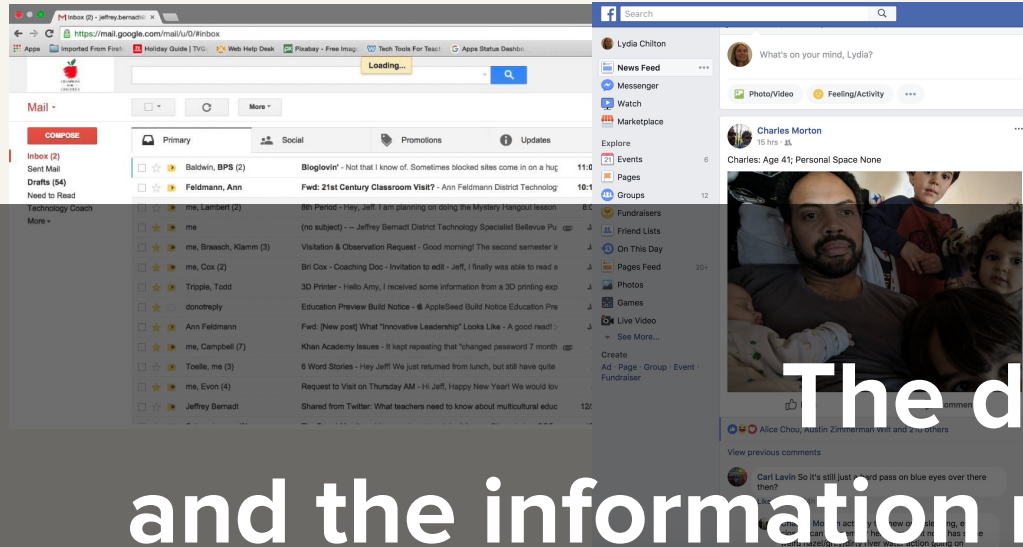
Delete Cancel

4170

 Delete event



From a back-end perspective Facebook and GMail are very similar. However, there is one difference...



The data is different,
and the information needs they serve are different.



You can drive a lot of human behavior with one database update



COMS 4170 · SPRING 2026

Rendering Templates

Dynamically Generating Web pages from Database Content



Internet Movie Database (IMDb) is a database of movies.



Menu

All



IMDbPro

Watchlist

layne-celeste

EN

Top 10 on IMDb this week >



#1

Wuthering Heights

2026 2h 16m R

★ 6.3 (23K) ☆ Rate

👁 Mark as watched

A passionate and tumultuous love story set against the backdrop of the Yorkshire moors, exploring the intense and destructive relationship...



#2

A Knight of the Seven Kingdoms

2026- 7 eps TV-MA

★ 9.0 (85K) ☆ Rate

👁 Mark as watched

A century before GOT, Ser Duncan the Tall, and his squire, Egg, wandered through Westeros while the Targaryen dynasty ruled the Iron Throne, and dragons were still remembered. Great destinies and enemies await the incomparable friends.



#3

The Housemaid

2025 2h 11m R

★ 6.8 (90K) ☆ Rate

👁 Mark as watched

A struggling young woman is relieved by the chance for a fresh start as a maid for a wealthy couple. Soon, she discovers that the family's secrets are far more dangerous than her own.



#4



#5



#6



#7



#8

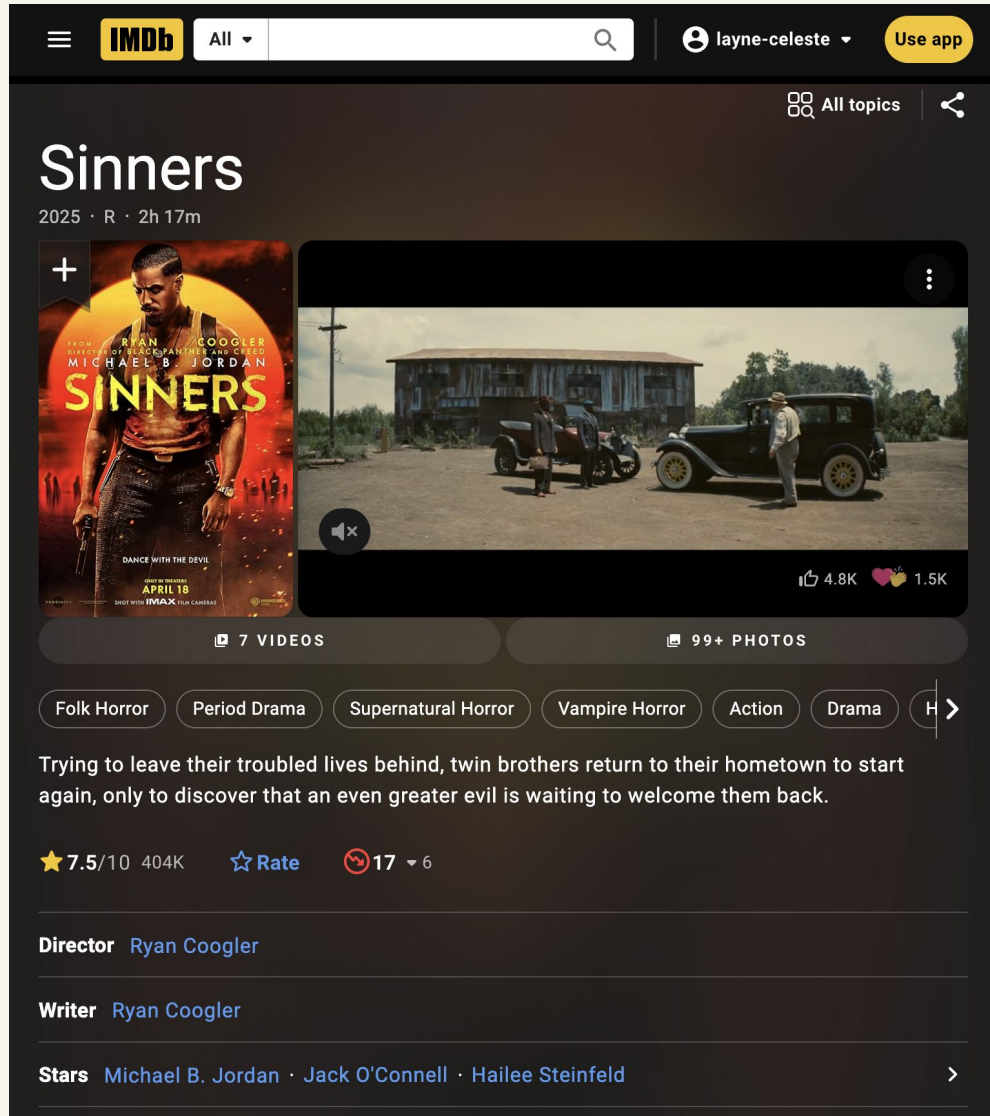


#9



#10

What goal are most people trying to achieve on IMDB?



Look up movie information:

- Who is the actor?
- What is the movie rating?
- What year was it released?
- Who is the director and what other movies have they done?
- etc

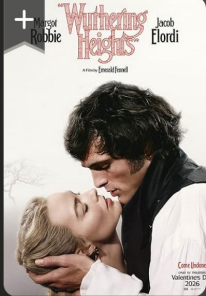


On IMDB, are there 100,000 static HTML pages sitting around?

 All 

layne-celeste 

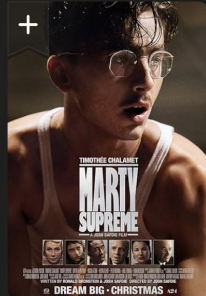
All topics 



 All 

layne-celeste 

All topics 



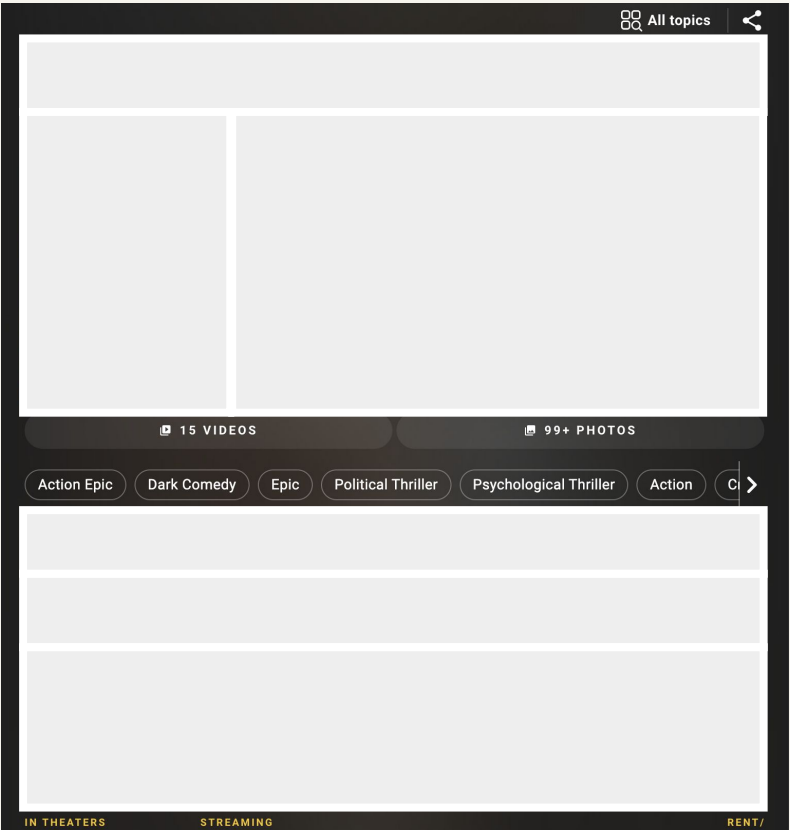
 All 

layne-celeste 

All topics 



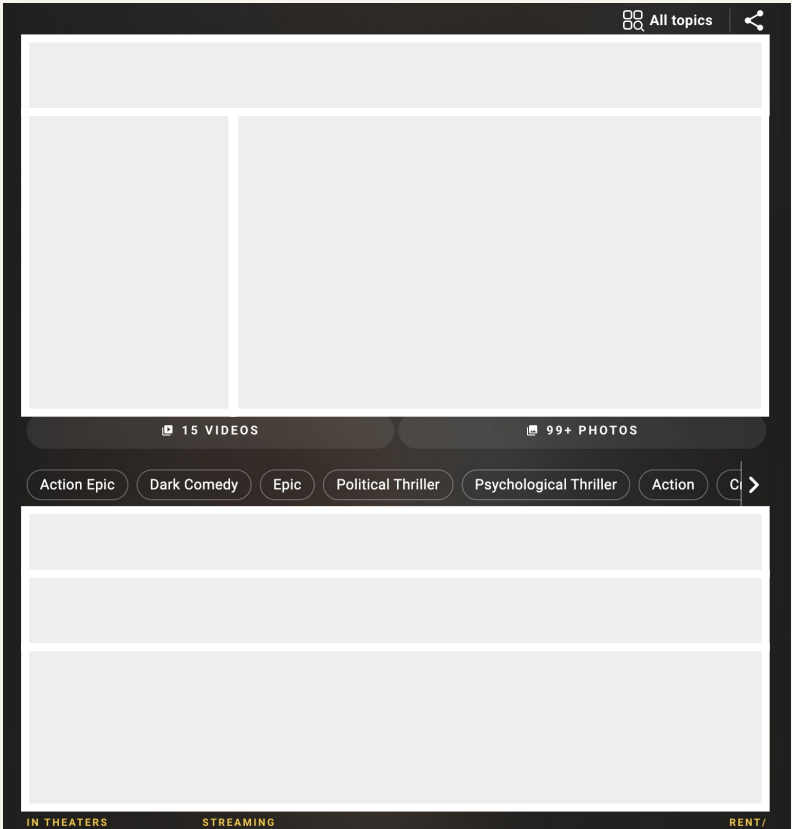
IMDB uses a template and database



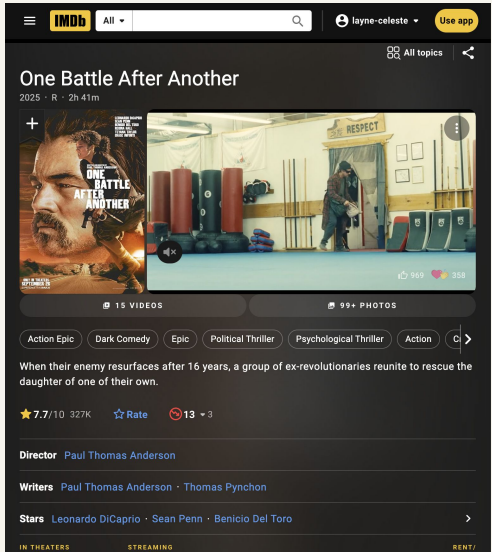
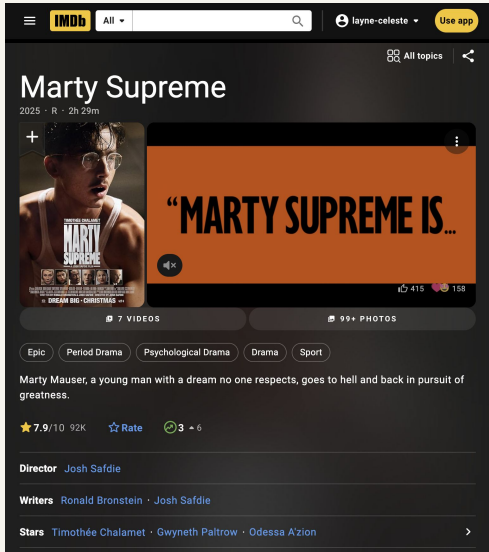
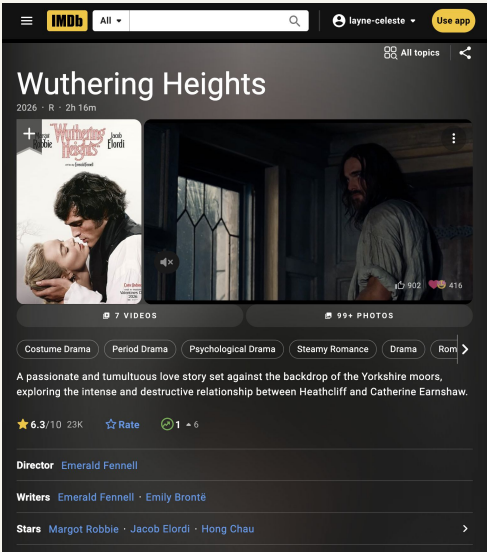
Title	Plot Summary	Poster	Year
One Battle After Another	When their enemy resurfaces after...	one-battle.jpg	2025
Wuthering Heights	A passionate and tumultuous love...	wuthering.jpg	2026
Marty Supreme	Marty Mauser, a young man with a dream...	marty-supreme.jpg	2025



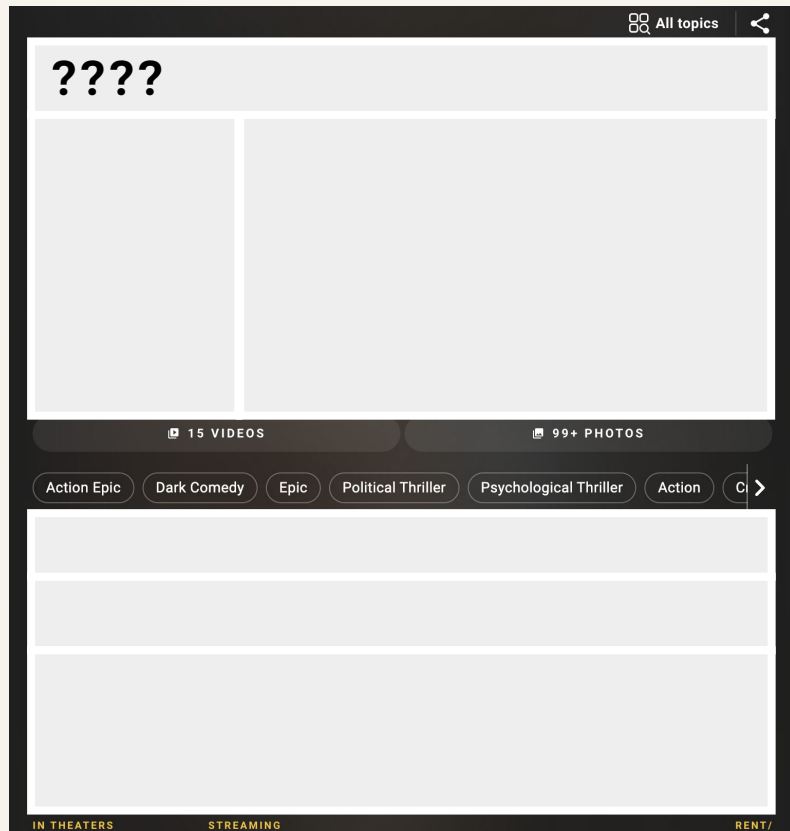
IMDB uses a template and database...and fills in the data dynamically on page load



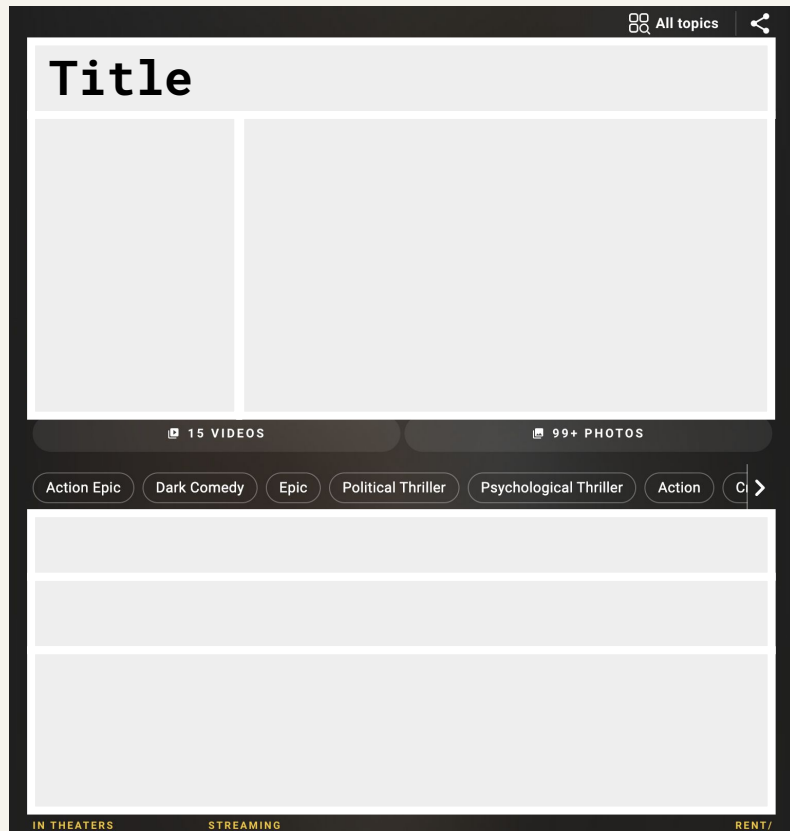
Title	Plot Summary	Poster	Year
One Battle After Another	When their enemy resurfaces after...	one-battle.jpg	2025
Wuthering Heights	A passionate and tumultuous love...	wuthering.jpg	2026
Marty Supreme	Marty Mauser, a young man with a dream...	marty-supreme.jpg	2025



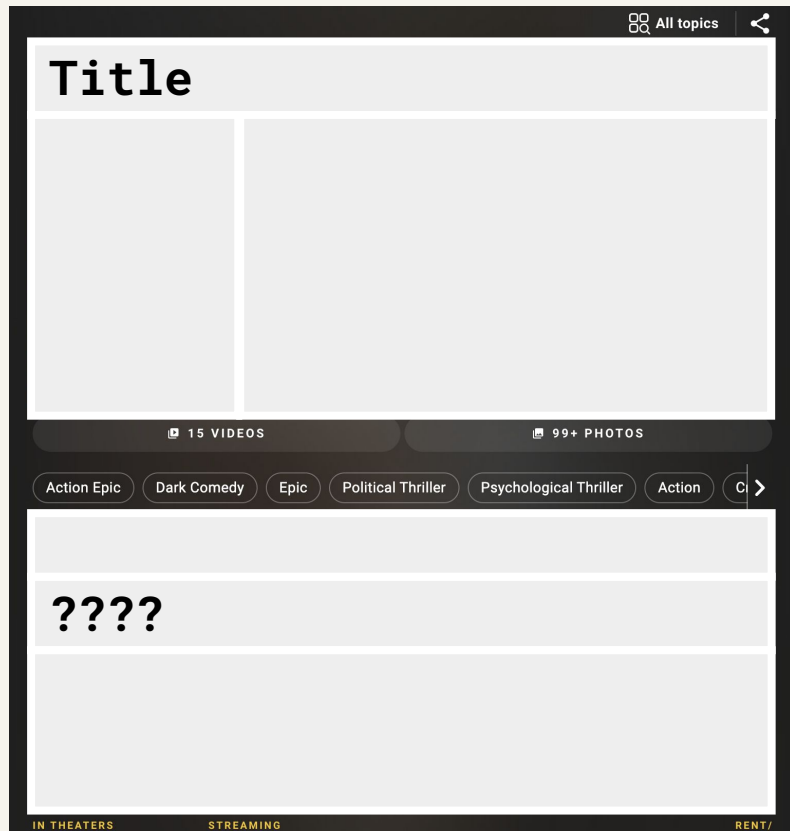
What goes in the template?



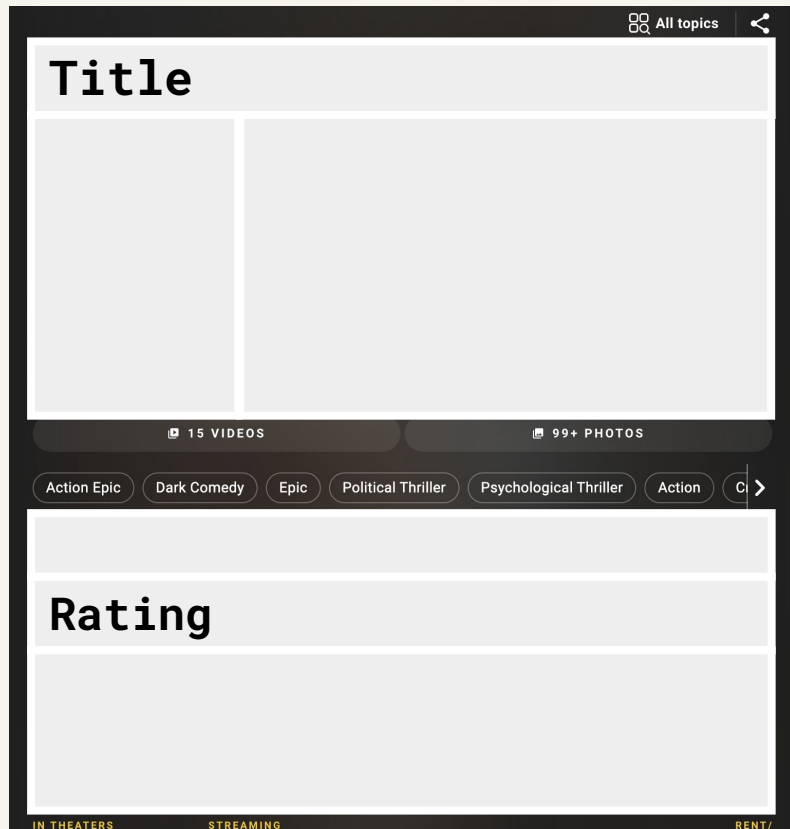
What goes in the template?



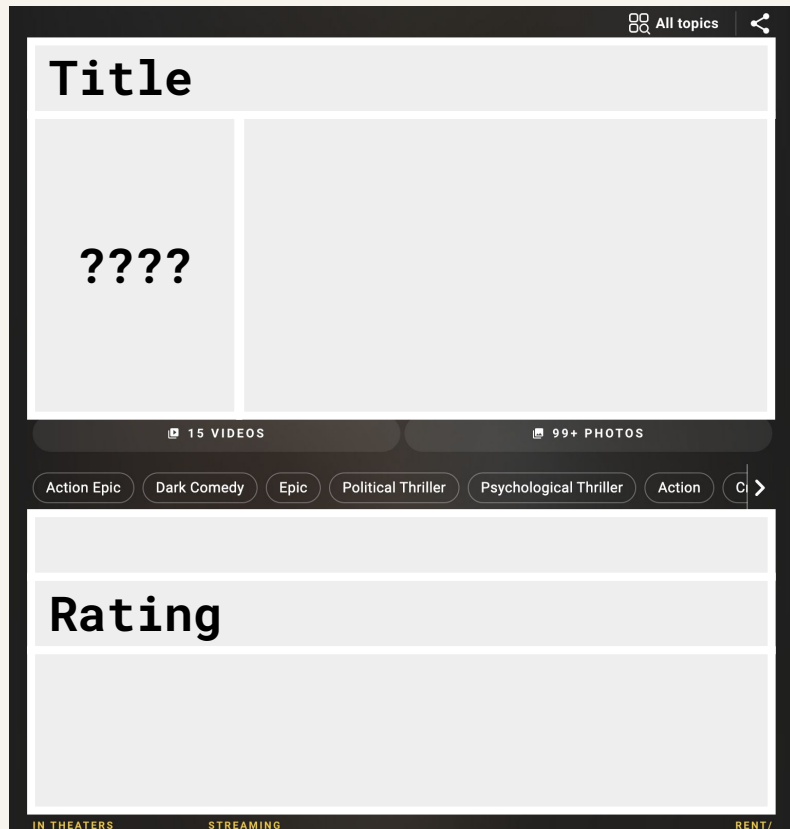
What goes in the template?



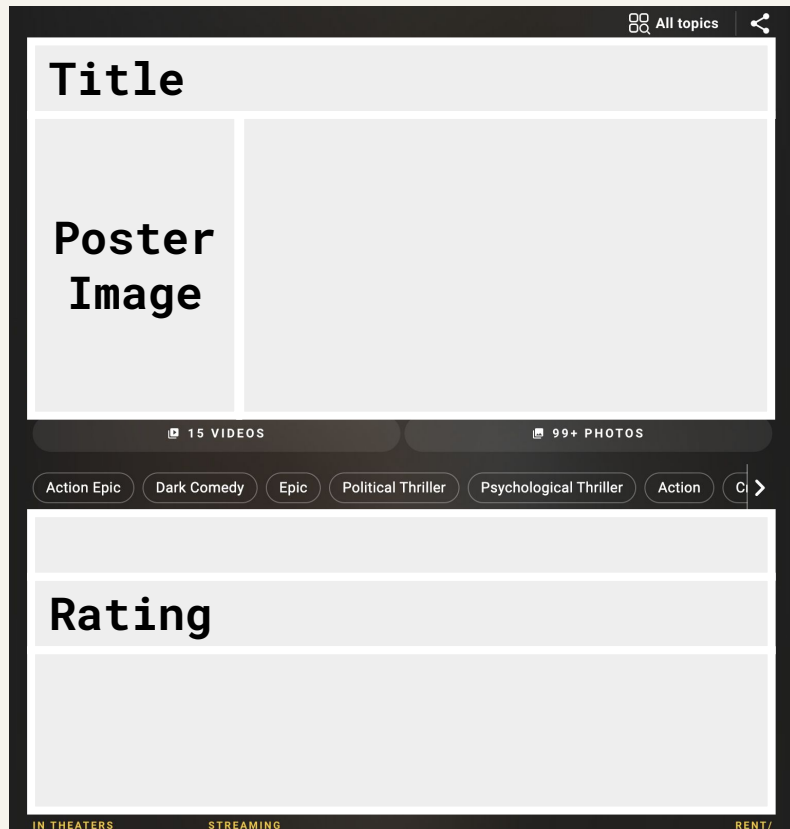
What goes in the template?



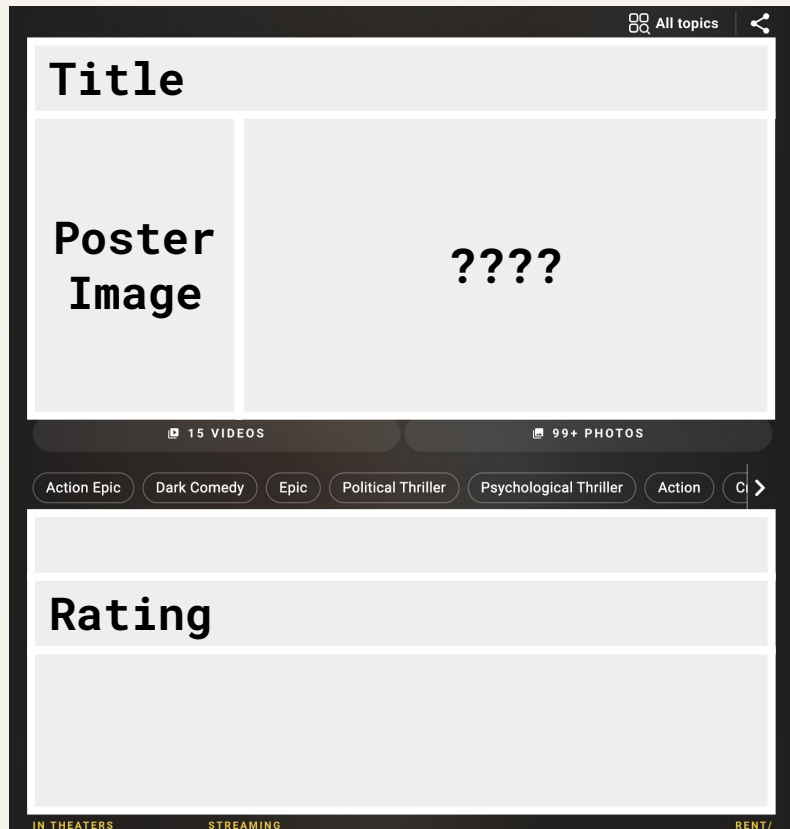
What goes in the template?



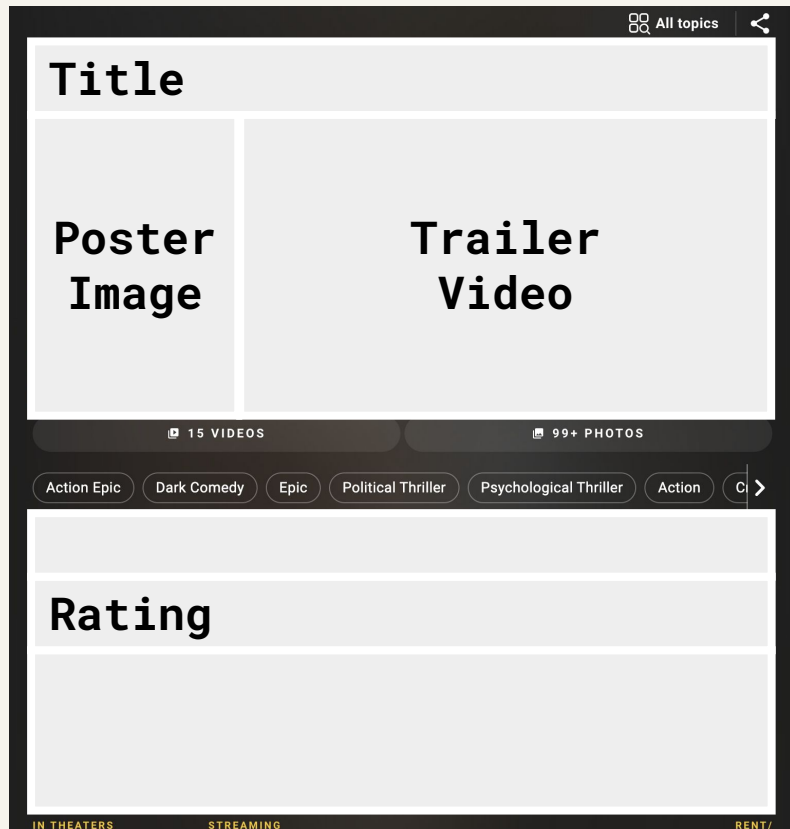
What goes in the template?



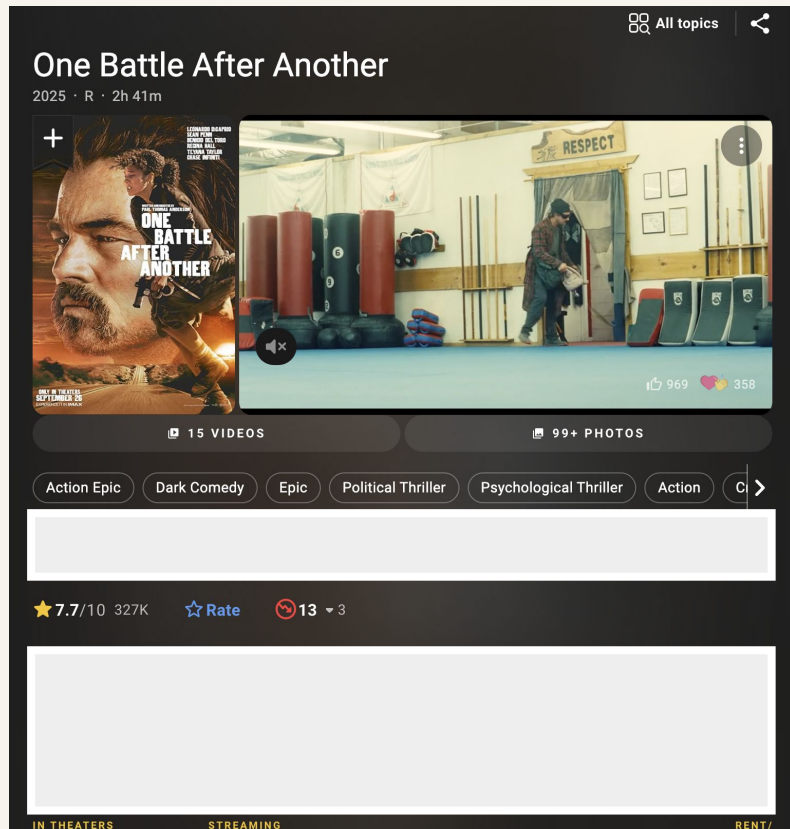
What goes in the template?



What goes in the template?



IMDB uses a template and dynamically fills the template from a database query



Anyone remember the other categories?

All topics

One Battle After Another

2025 · R · 2h 41m

+





969

358

15 VIDEOS

99+ PHOTOS

Action Epic

Dark Comedy

Epic

Political Thriller

Psychological Thriller

Action

C

★ 7.7/10 327K

☆ Rate

🔥 13 3

IN THEATERS

STREAMING

RENT/



Anyone remember the other categories?

All topics

One Battle After Another

2025 · R · 2h 41m

+

ONE BATTLE AFTER ANOTHER

15 VIDEOS

969

358

99+ PHOTOS

Action Epic

Dark Comedy

Epic

Political Thriller

Psychological Thriller

Action

C

Plot Summary

★7.7/10 327K

☆Rate

🔥13 3

Main Creatives

IN THEATERS

STREAMING

RENT/



Anyone remember the other categories?

All topics

One Battle After Another

2025 · R · 2h 41m

+

15 VIDEOS

99+ PHOTOS

Action Epic

Dark Comedy

Epic

Political Thriller

Psychological Thriller

Action

C

When their enemy resurfaces after 16 years, a group of ex-revolutionaries reunite to rescue the daughter of one of their own.

★ 7.7/10 327K

☆ Rate

🔥 13 3

Director

Paul Thomas Anderson

Writers

Paul Thomas Anderson · Thomas Pynchon

Stars

Leonardo DiCaprio · Sean Penn · Benicio Del Toro

IN THEATERS

STREAMING

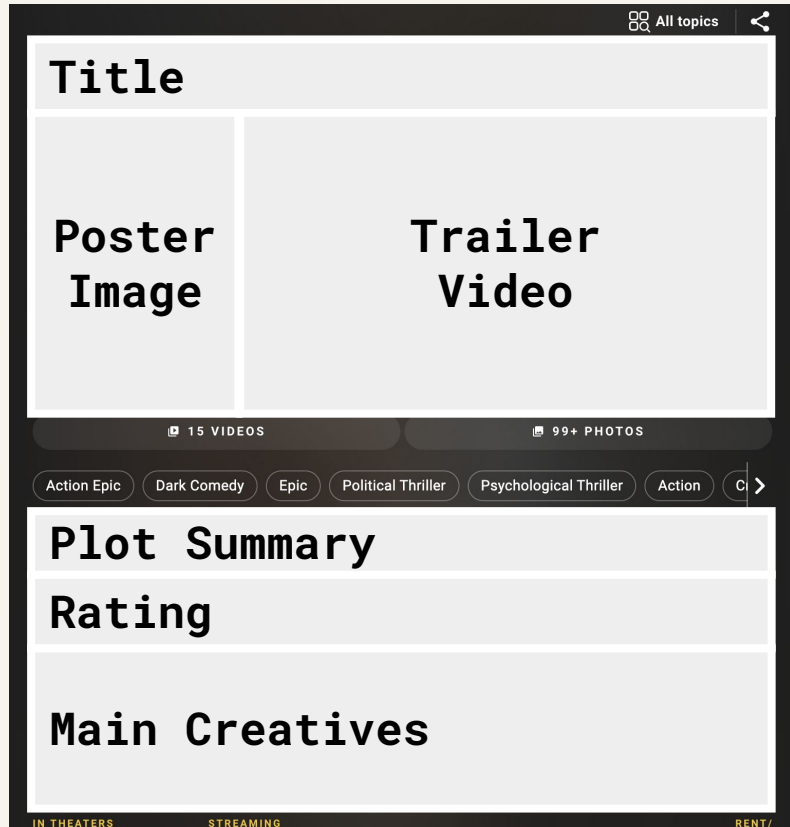
RENT/



IMDB Template



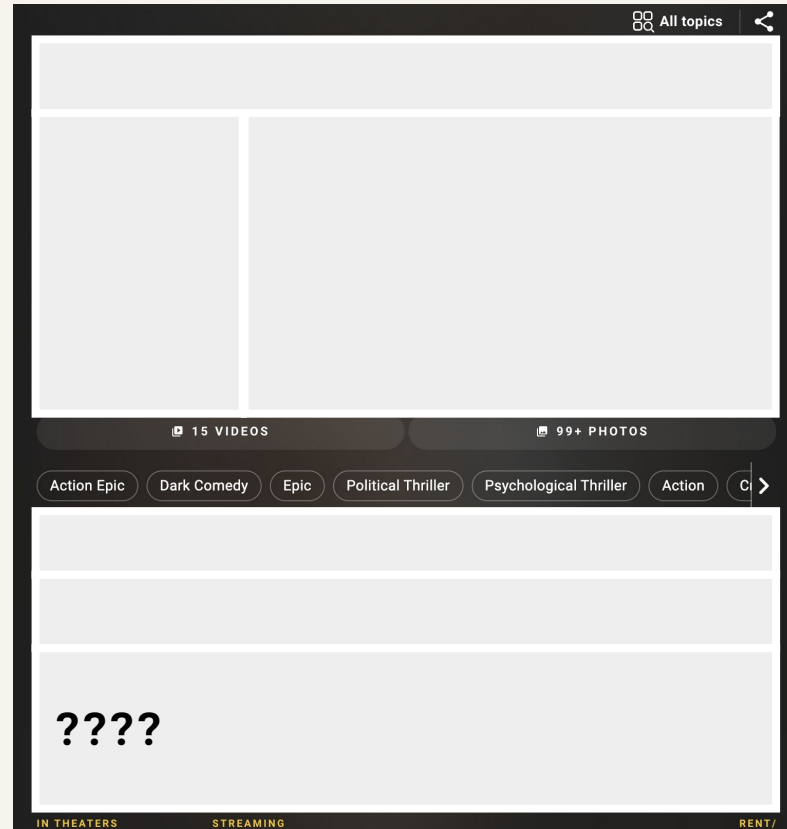
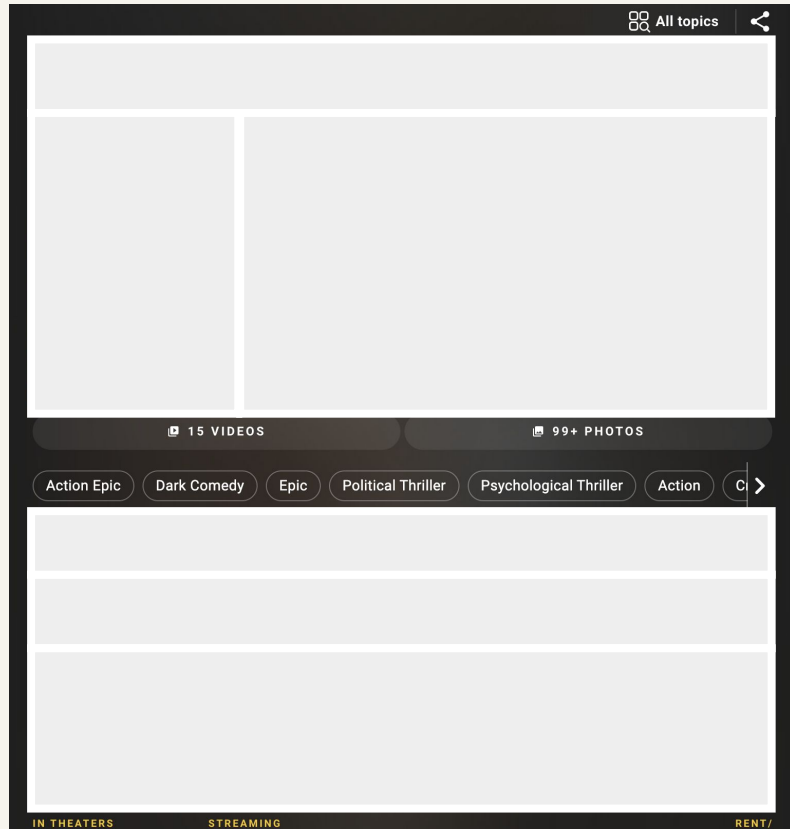
Templates are standardizations. What can go wrong?



Pros and cons of standardization



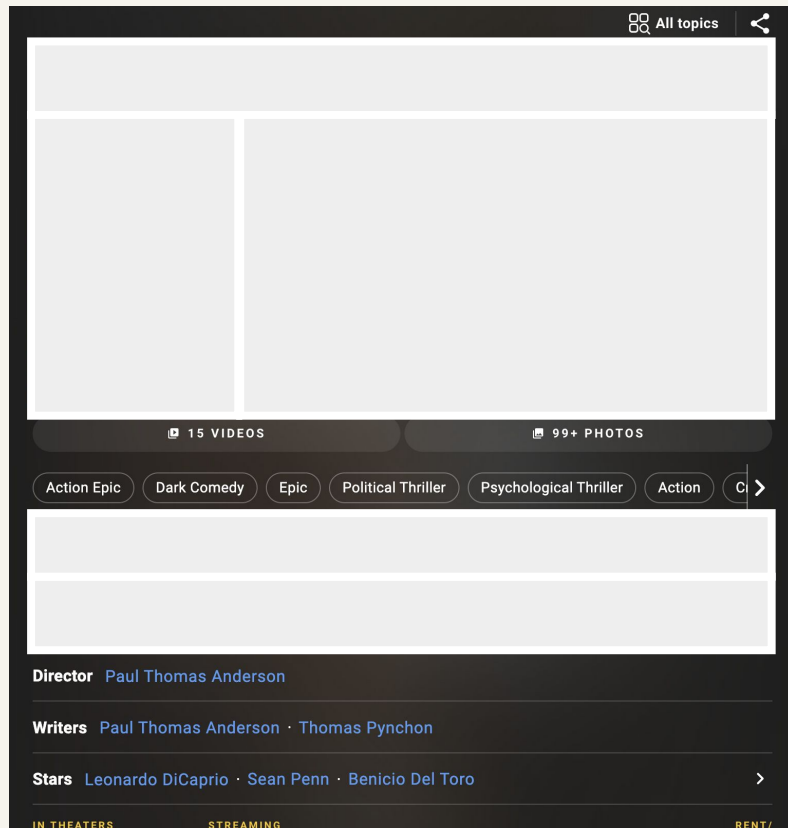
How should the template display the creatives?



You can have multiple directors...

...multiple writers and multiple stars.

These all need to be a list.



Standardization can be difficult.

Expect to iterate on your templates

All layne-celeste

All topics

Wuthering Heights

2026 · R · 2h 16m



7 VIDEOS 99+ PHOTOS

Costume Drama Period Drama Psychological Drama Steamy Romance Drama Rom >

A passionate and tumultuous love story set against the backdrop of the Yorkshire moors, exploring the intense and destructive relationship between Heathcliff and Catherine Earnshaw.

★ 6.3/10 23K ☆ Rate 🌱 1 ▲ 6

Director [Emerald Fennell](#)

Writers [Emerald Fennell](#) · [Emily Brontë](#)

Stars [Margot Robbie](#) · [Jacob Elordi](#) · [Hong Chau](#) >

All layne-celeste

All topics

Marty Supreme

2025 · R · 2h 29m



7 VIDEOS 99+ PHOTOS

Epic Period Drama Psychological Drama Drama Sport

Marty Mauser, a young man with a dream no one respects, goes to hell and back in pursuit of greatness.

★ 7.9/10 92K ☆ Rate 🌱 3 ▲ 6

Director [Josh Safdie](#)

Writers [Ronald Bronstein](#) · [Josh Safdie](#)

Stars [Timothée Chalamet](#) · [Gwyneth Paltrow](#) · [Odessa A'zion](#) >

All layne-celeste

All topics

One Battle After Another

2025 · R · 2h 41m



15 VIDEOS 99+ PHOTOS

Action Epic Dark Comedy Epic Political Thriller Psychological Thriller Action C >

When their enemy resurfaces after 16 years, a group of ex-revolutionaries reunite to rescue the daughter of one of their own.

★ 7.7/10 327K ☆ Rate 🌱 13 ▼ 3

Director [Paul Thomas Anderson](#)

Writers [Paul Thomas Anderson](#) · [Thomas Pynchon](#)

Stars [Leonardo DiCaprio](#) · [Sean Penn](#) · [Benicio Del Toro](#) >

IN THEATERS STREAMING RENT/



COMS 4170 · SPRING 2026

Implementing Templates in Flask





Jinja is a modern and designer-friendly templating language for Python, modelled after Django's templates. It is fast, widely used and secure with the optional sandboxed template execution environment:

```
<title>{% block title %}{% endblock %}</title>
<ul>
{% for user in users %}
  <li><a href="{{ user.url }}">{{ user.username }}</a></li>
{% endfor %}
</ul>
```

Features:

- sandboxed execution
- powerful automatic HTML escaping system for XSS prevention
- template inheritance
- compiles down to the optimal python code just in time
- optional ahead-of-time template compilation
- easy to debug. Line numbers of exceptions directly point to the correct line in the template.
- configurable syntax

Contents:

- [Introduction](#)
 - [Prerequisites](#)
 - [Installation](#)
 - [Basic API Usage](#)

Template Inheritance

The most powerful part of Jinja is template inheritance. Template inheritance allows you to build a base “skeleton” template that contains all the common elements of your site and defines **blocks** that child templates can override.

Sounds complicated but is very basic. It’s easiest to understand it by starting with an example.

Base Template

This template, which we’ll call `layout.html`, defines a simple HTML skeleton document that you might use for a simple two-column page. It’s the job of “child” templates to fill the empty blocks with content:

```
<!doctype html>
<html>
  <head>
    {% block head %}
      <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
      <title>{% block title %}{% endblock %} - My Webpage</title>
    {% endblock %}
  </head>
  <body>
    <div id="content">{% block content %}{% endblock %}</div>
    <div id="footer">
      {% block footer %}
        &copy; Copyright 2010 by <a href="http://domain.invalid/">you</a>.
      {% endblock %}
    </div>
  </body>
</html>
```

In this example, the `{% block %}` tags define four blocks that child templates can fill in. All the *block* tag does is tell the template engine that a child template may override those portions of the template.

Rendering a template with data

```
# import flask
from flask import Flask, url_for

# import jinja template engine
from flask import render_template

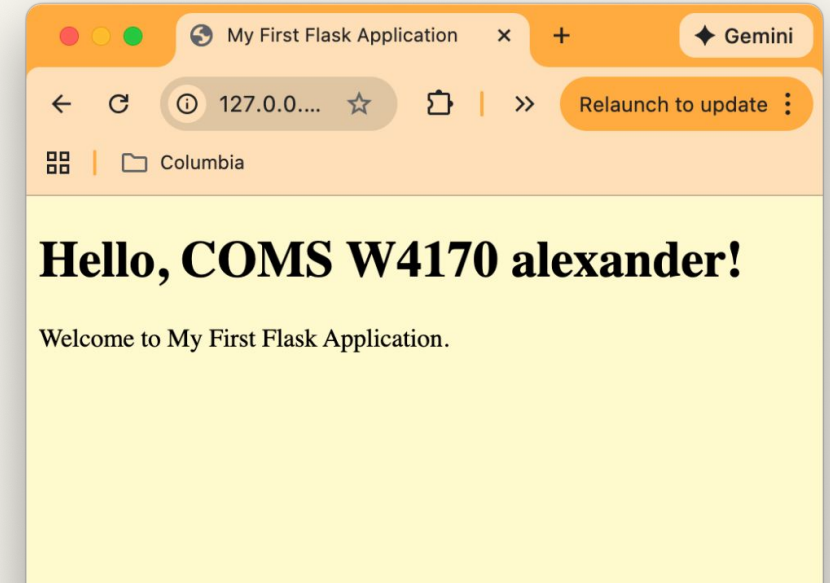
# create an app instance
app = Flask(__name__)

# create a base route
@app.route('/')

# create home method
def home():
    return render_template('index.html')

# create a welcome route
@app.route('/welcome/<name>')

# create welcome method
def welcome(name=None):
    return render_template('welcome.html', name=name)
```



```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Document</title>
</head>
<body>
  <h1>Welcome to COMS 4170, {{ name }}</h1>
</body>
</html>
```



You can also send objects to templates

```
books = {
    "001": {
        "id": "001",
        "title": "To Kill a Mockingbird",
        "author": "Harper Lee",
        "year": 1960,
        "rating": 4.8,
        "genres": ["Classic", "Southern Gothic", "Legal Drama"],
        "reviews": [
            "A profound exploration of racial injustice in the",
            "Scout's voice is one of the most memorable in all",
            "Required reading that actually deserves to be."
        ]
    },
    "002": { ... }
}
```

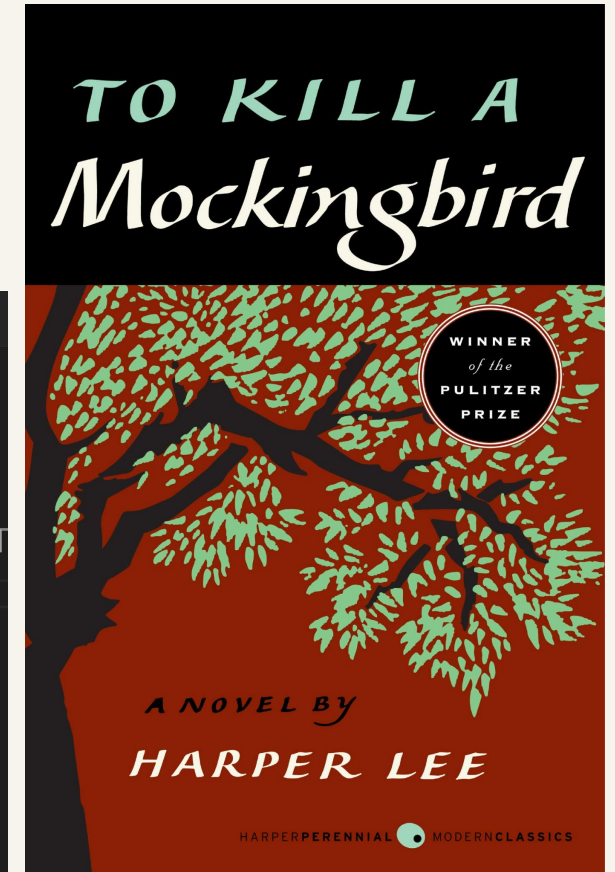
ROUTES

```
@app.route('/book/<int:id>')
def show_book(id=None):
    global books

    book = books[id]

    return render_template('book.html', book=book)
```

```
show.html
1 {% extends 'layout.html' %}
2
3 {% block content %}
4     <div class="row">
5         <p> These are book details {{book}}</p>
6     </div>
7
8 {% endblock content %}
```



New York Times Book List

These are book details – {'id': 3, 'title': 'To Kill a Mockingbird', 'author': 'Harper Lee', 'published_year': 1960}

and access data within the object

```
books = {
    "001": {
        "id": "001",
        "title": "To Kill a Mockingbird",
        "author": "Harper Lee",
        "year": 1960,
        "rating": 4.8,
        "genres": ["Classic", "Southern Gothic", "Legal Drama"],
        "reviews": [
            "A profound exploration of racial injustice in the American South",
            "Scout's voice is one of the most memorable in all of literature",
            "Required reading that actually deserves to be."
        ]
    },
    "002": { ... }
}
```

ROUTES

```
@app.route('/book/<int:id>')
def show_book(id=None):
    global books

    book = books[id]

    return render_template('book.html', book=book)
```

```
<> show.html X
1  {% extends 'layout.html' %}
2
3  {% block content %}
4      <div class="row">
5          <p> Book name: {{book.title}}</p>
6      </div>
7  {% endblock content %}
```

New York Times Book List

Book name: To Kill a Mockingbird



and access that data multiple times

```
books = {
    "001": {
        "id": "001",
        "title": "To Kill a Mockingbird",
        "author": "Harper Lee",
        "year": 1960,
        "rating": 4.8,
        "genres": ["Classic", "Southern Gothic", "Legal Drama"],
        "reviews": [
            "A profound exploration of racial injustice in the American South.",
            "Scout's voice is one of the most memorable in all of literature.",
            "Required reading that actually deserves to be."
        ]
    },
    "002": { ... }
}
```

ROUTES

```
@app.route('/book/<int:id>')
def show_book(id=None):
    global books

    book = books[id]

    return render_template('book.html', book=book)
```

```
show.html x
1  {% extends 'layout.html' %}
2
3  {% block content %}
4      <div class="row">
5
6          <p> Book name: {{book.title}}</p>
7          <div>
8              <p> Author: {{book.author}}</p>
9              <p> Publication date: {{book.published_year}}</p>
10             </div>
11
12          </div>
13  {% endblock content %}
```

New York Times Book List

Book name: To Kill a Mockingbird

Author: Harper Lee

Publication date: 1960

Templates are a great way to show all the information of a database element; but what if we want to show all the items

```
books = {
    "001": {
        "id": "001",
        "title": "To Kill a Mockingbird",
        "author": "Harper Lee",
        "year": 1960,
        "rating": 4.8,
        "genres": ["Classic", "Southern Gothic", "Legal Drama"],
        "reviews": [
            "A profound exploration of racial injustice in the American South.",
            "Scout's voice is one of the most memorable in all of literature.",
            "Required reading that actually deserves to be."
        ]
    },
    "002": { ... }
}
```

ROUTES

```
@app.route('/book/<int:id>')
def show_book(id=None):
    global books

    book = books[id]

    return render_template('book.html', book=book)
```

```
4 <div class="row">
5
6 <p> Book name: {{book.title}}</p>
7
8 <div>
9     <p> Author: {{book.author}}</p>
10    <p> Publication date: {{book.published_year}}</p>
11 </div>
12 <div>
13     Reviews:|
14     <ul>
15         {% for review in book.reviews %}
16         <li>{{ review }}</li>
17         {% endfor %}
18     </ul>
19 </div>
20
21 {% endblock content %}
```

New York Times Book List

Book name: To Kill a Mockingbird

Author: Harper Lee

Publication date: 1960

Reviews:

- A profound exploration of racial injustice in the American South.
- Scout's voice is one of the most memorable in all of literature.
- Required reading that actually deserves to be.



...what if we want to return all the books with a classic genre

```
books = {
    "001": {
        "id": "001",
        "title": "To Kill a Mockingbird",
        "author": "Harper Lee",
        "year": 1960,
        "rating": 4.8,
        "genres": ["Classic", "Southern Gothic", "Legal Drama"],
        "reviews": [
            "A profound exploration of racial injustice in the American So",
            "Scout's voice is one of the most memorable in all of literatu",
            "Required reading that actually deserves to be."
        ]
    },
    "002": { ... }
}
```

ROUTES

```
@app.route('/book/<int:id>')
```

```
def show_book(id=None):
    global books
```

```
    book = books[id]
```

```
    return render_template('book.html', book=book)
```

1. Give the route a search term rather than a name or id:

```
@app.route('/search/<term>')
def search_books(term):
    global books
```

2. Iterate over all of the books and find matching ones
3. Return an array of results (not just one result)
4. You could display the array of results using a template; but you could also dynamically create the list in JavaScript.



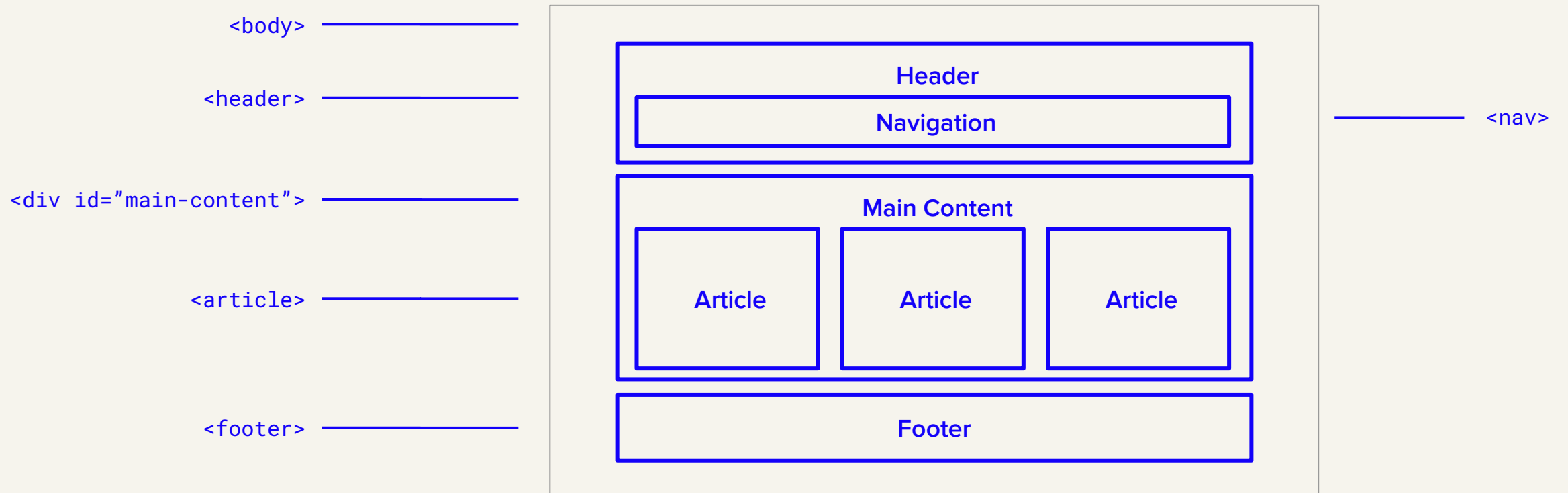
Base layout.html with Bootstrap added

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta http-equiv="X-UA-Compatible" content="IE=edge">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>Oscar Winning Movie Database</title>
8
9      <!-- Bootstrap CSS -->
10     <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0-beta1/dist/css/bootstrap.min.css"
11         rel="stylesheet" integrity="sha384-giJF6kkoqN00vy+HMDP7az0uL0xtbfIcaT9wjKHr8RbDVddVHyTfAAsrekwKmp1"
12         crossorigin="anonymous">
13
14 </head>
15 <body>
16
17     <!-- jQuery CDN -->
18     <script src="https://code.jquery.com/jquery-1.12.4.js"></script>
19
20     <!-- bootstrap CDN -->
21     <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0-beta1/dist/js/bootstrap.min.js"
22         integrity="sha384-pQQkAEnwaBkjpqZ8RU1fF1AKtTcHJwFl3pblpTLHXybJjHpMYo79HY3hIi4NKxyj"
23         crossorigin="anonymous"></script>
24
25 </body>
26 </html>
```



Template Basics

Structuring Base Layout



Base Template

```
7 <title>Oscar Winning Movie Database</title>
8
9 <!-- Bootstrap CSS -->
10 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0-beta1/dist/css/bootstrap.m
11
12 </head>
13 <body>
14 <div class="container">
15 <div class="row">
16 <div class="nav navbar navbar-light bg-light">
17 <div class="container-fluid">
18 <span class="navbar-brand mb-0 h1">Oscar Winning Movie Database</span>
19 </div>
20 </div>
21 </div>
22 <!-- <div class="row"> -->
23 {% block content %}{% endblock %}
24 <!-- </div> -->
25 <div class="row">
26 <div class="nav navbar fixed-bottom navbar-light bg-light">
27 <div class="container-fluid">
28 <a class="navbar-brand" href="#">&copy; 2021 Columbia University</a>
29 </div>
30 </div>
31 </div>
32 </div>
33
34 <!-- jQuery -->
35 <script src="https://code.jquery.com/jquery-1.12.4.js"></script>
36
37 <!-- bootstrap CDN -->
38 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0-beta1/dist/js/bootstrap.m
39
```

Header

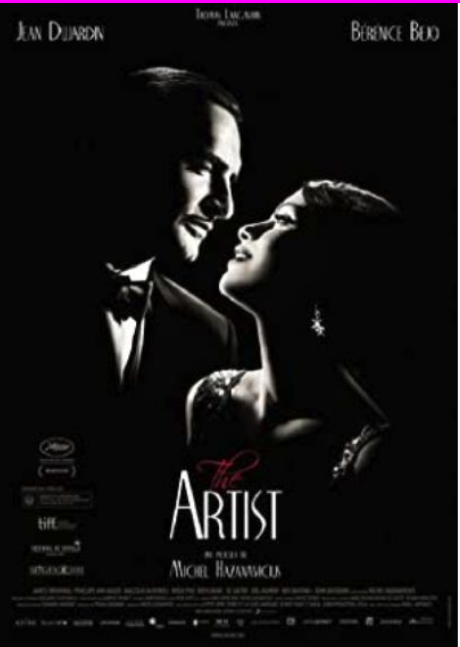
Content

Footer

Oscar Winning Movie Database

Search Movie Database

Type to search...



The Artist
Michel Hazanavicius

Delete



Argo
Ben Affleck

Delete

Base Template

```
7 <title>Oscar Winning Movie Database</title>
8
9 <!-- Bootstrap CSS -->
10 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0-beta1/dist/css/bootstrap.m
11
12 </head>
13 <body>
14 <div class="container">
15 <div class="row">
16 <nav class="navbar navbar-light bg-light">
17 <div class="container-fluid">
18 <span class="navbar-brand mb-0 h1">Oscar Winning Movie Database</span>
19 </div>
20 </nav>
21 </div>
22 <!-- <div class="row"> -->
23 {% block content %}{% endblock %}
24 <!-- </div> -->
25 <div class="row">
26 <nav class="navbar fixed-bottom navbar-light bg-light">
27 <div class="container-fluid">
28 <a class="navbar-brand" href="#">&copy; 2021 Columbia University</a>
29 </div>
30 </nav>
31 </div>
32 </div>
33
34 <!-- jQuery CDN -->
35 <script src="https://code.jquery.com/jquery-1.12.4.js"></script>
36
37 <!-- bootstrap CDN -->
38 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0-beta1/dist/js/bootstrap.m
39
```

Oscar Winning Movie Database

Search Movie Database



The Artist

Michel Hazanavicius

Delete



Argo

Ben Affleck

Delete

Child Template

Child Template

A child template might look like this:

```
{% extends "layout.html" %}
{% block title %}Index{% endblock %}
{% block head %}
    {{ super() }}
    <style type="text/css">
        .important { color: #336699; }
    </style>
{% endblock %}
{% block content %}
    <h1>Index</h1>
    <p class="important">
        Welcome on my awesome homepage.
    </p>
{% endblock %}
```

The `{% extends %}` tag is the key here. It tells the template engine that this template “extends” another template. When the template system evaluates this template, first it locates the parent. The `extends` tag must be the first tag in the template. To render the contents of a block defined in the parent template, use `{{ super() }}`.

Child Template

Syntax

`{% extends 'layout.html' %}`

`{% block content %}`

`{% endblock content %}`

```
1 {% extends 'layout.html' %}
2
3 {% block content %}
4
5     <!-- search form -->
6     <div class="row">
7         <form>
8             <label for="exampleDataList" class="form-label">Search Movie Database</label>
9             <input class="form-control" list="datalistOptions" id="exampleDataList" placeholder="Type to search...">
10         </form>
11     </div>
12
13     <!-- list of movies -->
14     <div class="row movies"></div>
15
16 {% endblock content %}
```

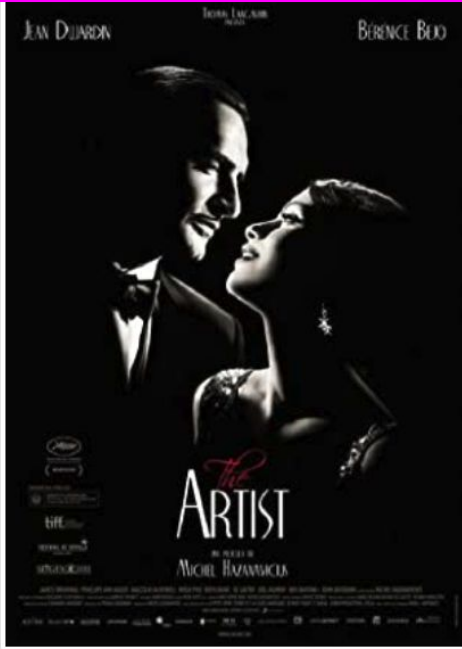
Search form

Container to render the list of movies

Oscar Winning Movie Database

Search Movie Database

Type to search...



The Artist
Michel Hazanavicius

Delete



Argo
Ben Affleck

Delete

12 Year
Steve M

Delete

© 2021 Columbia University

Child Template

```
1 {% extends 'layout.html' %}  
2  
3 {% block content %}  
4  
5     <!-- search form -->  
6     <div class="row">  
7         <form>  
8             <label for="exampleDataList" class="form-label">Search Movie Database</label>  
9             <input class="form-control" list="datalistOptions" id="exampleDataList" placeholder="Type to search...">  
10        </form>  
11    </div>  
12  
13    <!-- list of movies -->  
14    <div class="row movies"></div>  
15  
16 {% endblock content %}
```

Tells the template engine
that this template
“extends” another template

Tells the template engine
that a child template may
override those portions of
the template



COMS 4170 · SPRING 2026

Homework 06



Homework 06

Warm Up due **Friday at 11:59PM**

- You can complete this now

Main Assignment due **Monday at 11:59PM**

- You will need Tuesday's lecture

Homework is posted to Courseworks:
<https://courseworks2.columbia.edu/courses/237745>



Homework 06: Search Application Functionality & Usability

Create a webpage that lets users interact with data. The basic functionality your site must allow is to:

- **Search data (all resources)**
- Create data (a resource)
- **Read data (all and one resource(s))**
- Update data (a resource)
- Delete data (a resource)



Pick a dataset and put 10 items in it by hand.

NBA Players **Averaging** 25+ Points in 2025



Makeup **under** \$25 for acne-prone skin



Best Picture Winners **After** 2000



Study Spots **Near** Columbia



Independent Coffee Shops **in** my hometown



Indie Albums Released **After** 2020



Have a user need in mind. Who needs this data? Why?

Family-owned Ice cream parlors in the US



A college student is graduating and wants to road trip from NYC to Florida. Along the way, they want to find top-rated family-owned ice cream parlors so they can write about it for their travel blog.



Specific needs are easier to design for.

Mobility Exercises

2026 Fantasy Football Rankings

NYC City Budget Allocations

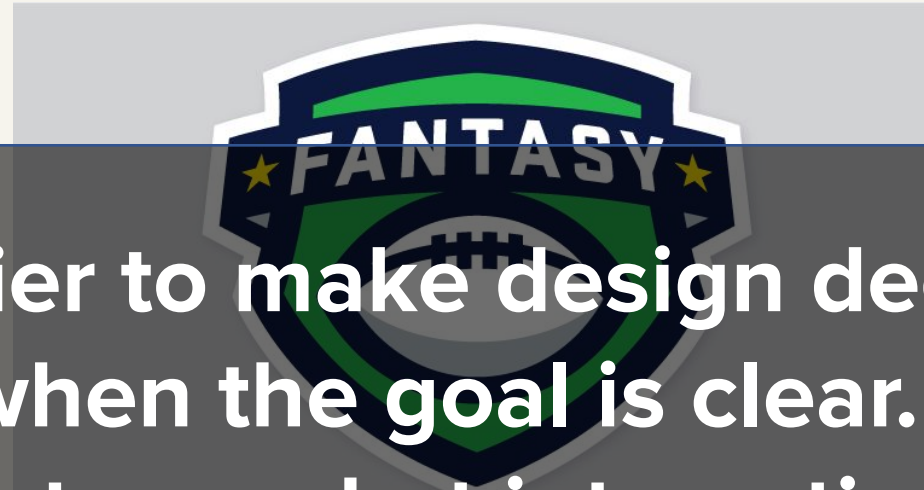
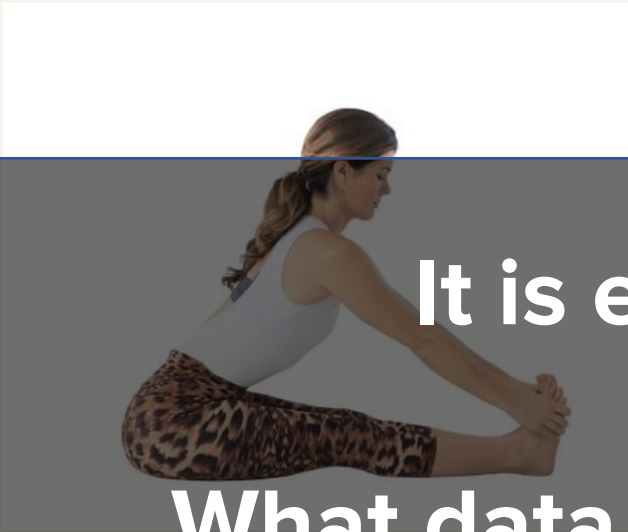
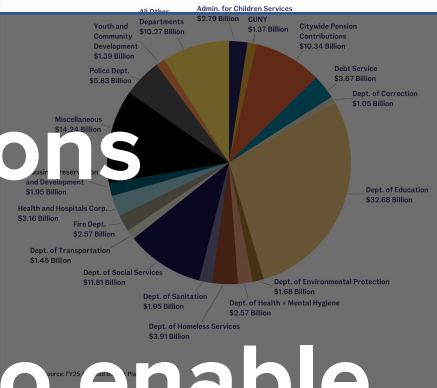


Figure 1: Fiscal Year 2025 Adopted Budget Expense Funding by Major Department or Expense Category



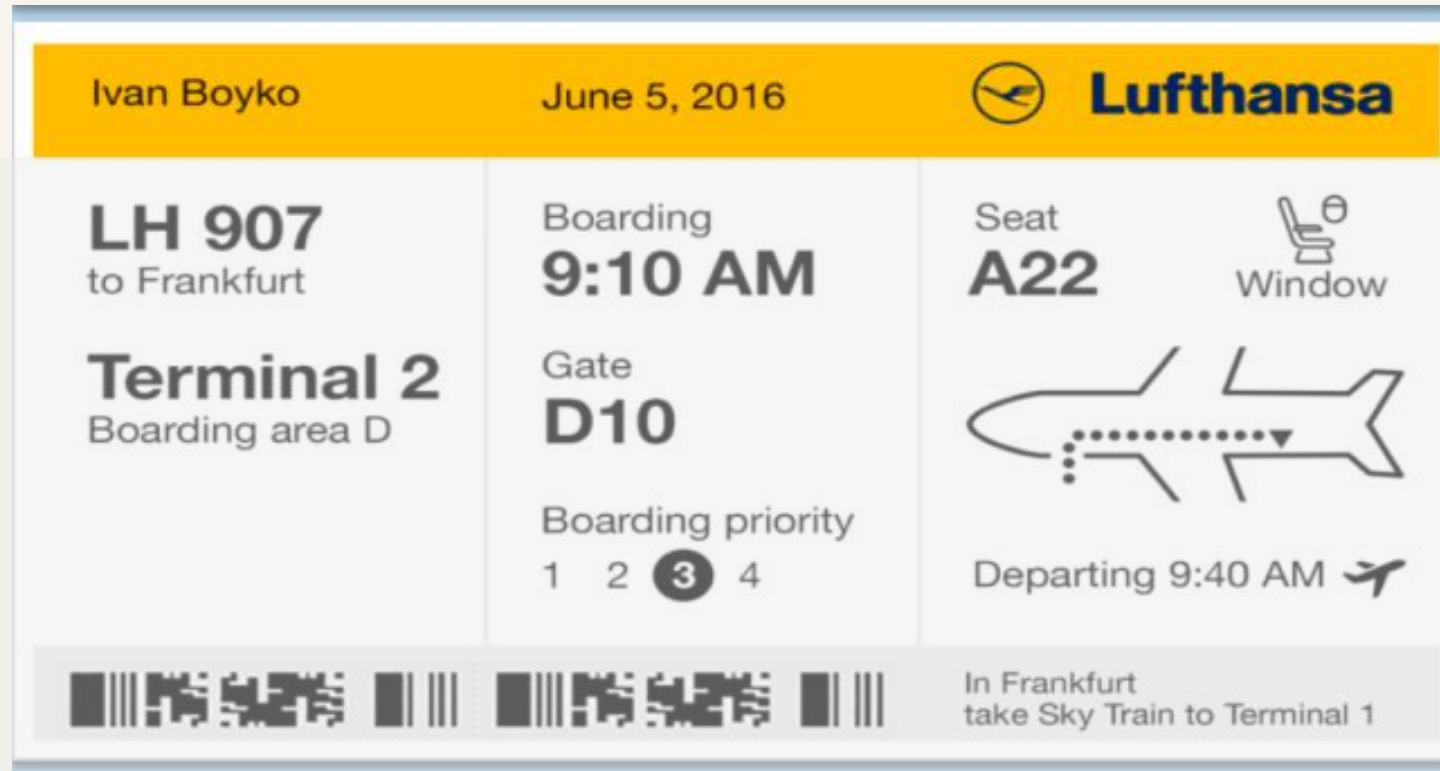
A person working from home who wants to relieve lower back stiffness.

A fantasy football player deciding between two wide receivers.

A student comparing how much different districts allocate to public schools



Remember, use good design principles, and know what the user goal is



Who is the user?

Someone taking a flight to Frankfurt

What is the users' primary goal?

To get through the airport to their seat on the plane

Homework 06: Enable search over data and viewing individual data elements (like IMDB)

Create a webpage that lets users interact with data. The basic functionality your site must allow is to:

- **Search data (all resources)**
- Create data (a resource)
- **Read data (all and one resource(s))**
- Update data (a resource)
- Delete data (a resource)

It doesn't have to be pretty; just functional.



You can implement with **flask-starter-template** or **people.zip**

- When starting with another code base, be sure to do iterative programming
- Do not just delete code you think you might not need. It will definitely create serious errors.



Weird Flask behaviors to watch out for ...

TypeError

TypeError: Object of type Undefined is not JSON serializable

Traceback (most recent call last)

File "/Users/celeste/Documents/GitHub/COMS-W4170/week-06/flask-template-starter-code/venv/lib/python3.9/site-packages/flask/app.py", line 1536, in __call__

```
return self.wsgi_app(environ, start_response)
```

File "/Users/celeste/Documents/GitHub/COMS-W4170/week-06/flask-template-starter-code/venv/lib/python3.9/site-packages/flask/app.py", line 1514, in wsgi_app

```
response = self.handle_exception(e)
```

File "/Users/celeste/Documents/GitHub/COMS-W4170/week-06/flask-template-starter-code/venv/lib/python3.9/site-packages/flask/app.py", line 1511, in wsgi_app

```
response = self.full_dispatch_request()
```

File "/Users/celeste/Documents/GitHub/COMS-W4170/week-06/flask-template-starter-code/venv/lib/python3.9/site-packages/flask/app.py", line 919, in full_dispatch_request

```
rv = self.handle_user_exception(e)
```

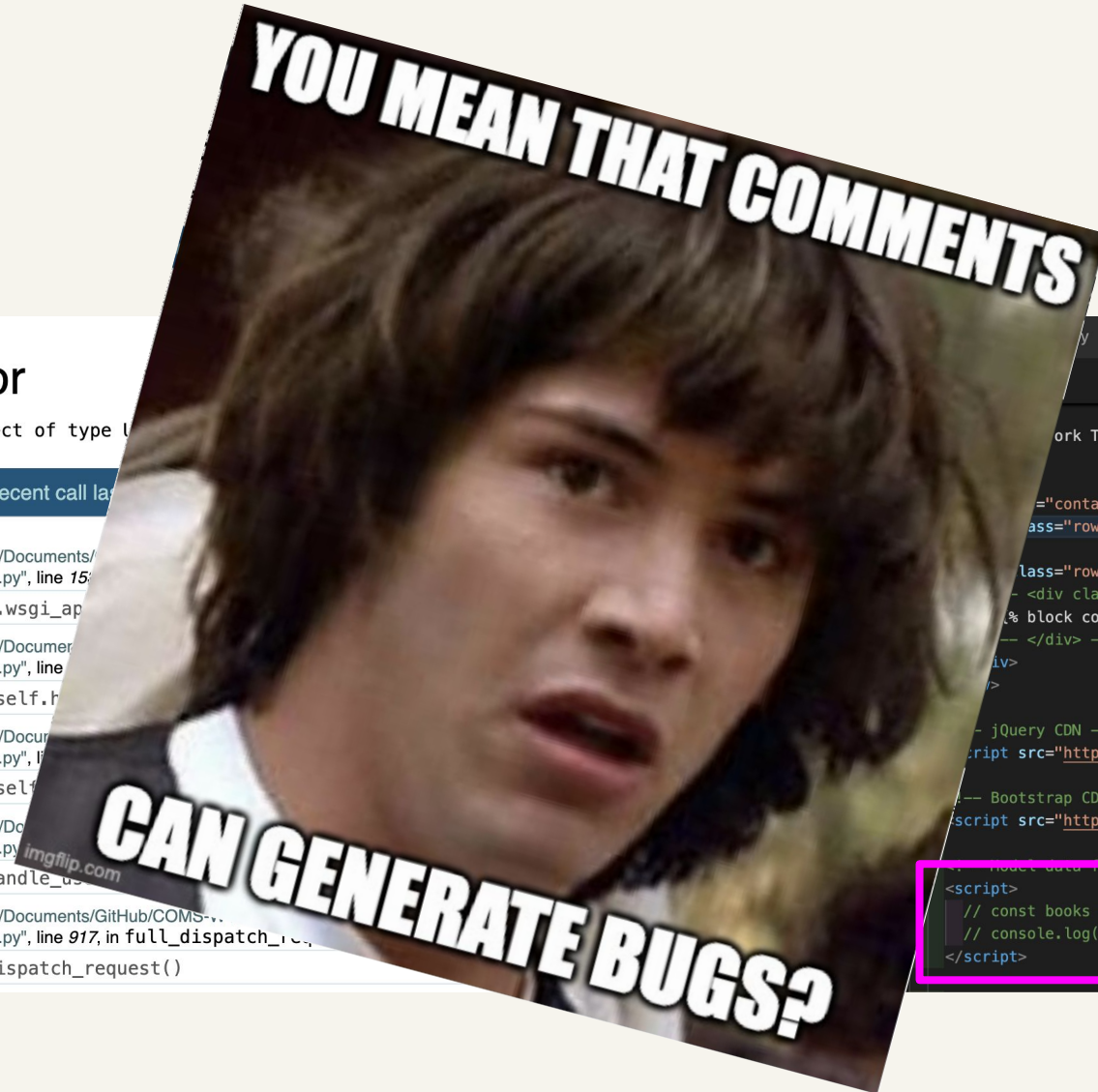
File "/Users/celeste/Documents/GitHub/COMS-W4170/week-06/flask-template-starter-code/venv/lib/python3.9/site-packages/flask/app.py", line 917, in full_dispatch_request

```
rv = self.dispatch_request()
```

```
<> show.html  server.py  <> layout.html X
2  <html lang="en">
3  <head>
10
11  <title>New York Times Book List</title>
12  </head>
13  <body>
14  <div class="container">
15  > <div class="row"> ...
25  </div>
26  <div class="row">
27  <!-- <div class="row books"> -->
28  <div class="row books">
29  <!-- </div> -->
30  </div>
31  </div>
32
33  <!-- jQuery CDN -->
34  <script src="https://code.jquery.com/jquery-1.12.4.js"></script>
35
36  <!-- Bootstrap CDN files -->
37  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0-beta1/dist/js/bootstrap.min.js" integrity="sha384-pQQkAEnwaBkjpqZ8RU1fF1AK"
38
39  <!-- Model data from server.py -->
40  <script>
41  // const books = JSON.parse('{{ books|tojson }}')
42  // console.log(books)
43  </script>
44
```



Let's comment it out, and try again ...



TypeError

TypeError: Object of type l

Traceback (most recent call last)

File "/Users/celeste/Documents/packages/flask/app.py", line 15:

```
return self.wsgi_ap
```

File "/Users/celeste/Documents/Python/packages/flask/app.py", line

```
response = self.h
```

File "/Users/celeste/Documents/Python/packages/flask/app.py", line 146, in __init__

```
response = self
```

File "/Users/celeste/Do
packages/flask/app.py

```
rv = self.handle_us
```

File "/Users/celeste/Documents/GitHub/COMS-...
packages/flask/app.py", line 917, in full_dispatch_req...

```
rv = self.dispatch_request()
```

layout.html

ork Times Book List</title>

```
= "container">
```

```
class="row"> ...
```

```
class="row">
```

```
% block content %}{% endblock %}
```

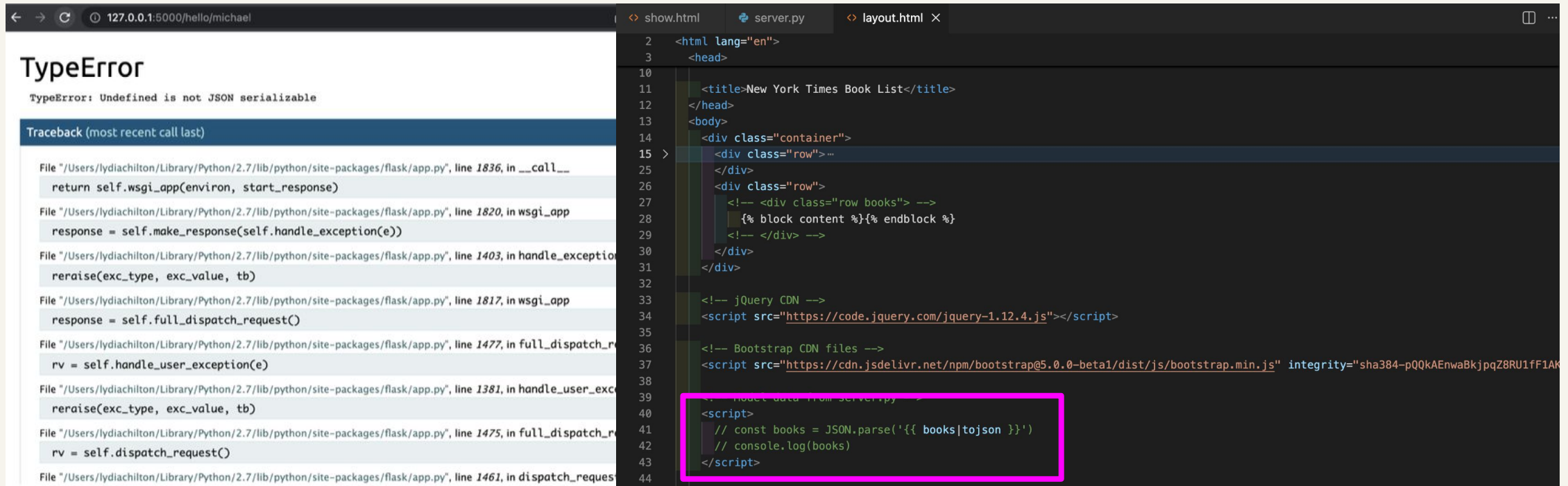
```
src="https://code.jquery.com/jquery-1.12.4.js"></script>
```

```
src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0-beta1/dist/js/bootstrap.min.js" integrity="sha384-pQQkAEnwaBkjpqZ8RU1fF1AK
```

```
get_data_from_server.py
```

```
<script>
  // const books = JSON.parse('{{ books|tojson }}')
  // console.log(books)
</script>
```

Even when commented out, Flask will still replace all the `{{}}` syntax in the template with data



The screenshot shows a web browser window on the left and a code editor on the right. The browser window displays a `TypeError` message: `TypeError: Undefined is not JSON serializable`. Below the message is a traceback showing the error occurred in `flask/app.py` at line 1836, in `__call__`, and line 1820, in `wsgi_app`. The code editor shows a template file `layout.html` with HTML and JavaScript code. A script tag is highlighted with a red box, containing the following code:

```
<script>
    // const books = JSON.parse('{{ books|tojson }}')
    // console.log(books)
</script>
```



Hint: How did I store my employees?

```
employees = {
    "laura":
        {
            "first_name": "Laura",
            "last_name": "Sutton",
            "role": "Regional Manager",
            "region": "Southwest",
        },
    "willow":
        {
            "first_name": "Willow",
            "last_name": "Madison",
            "role": "Assistant to the Regional Manager",
            "region": "Southwest",
        },
}

# ROUTES
@app.route('/book/<int:id>')
def show_book(id=None):
    global books

    book = books[id]

    return render_template('book.html', book=book)
```

Use a dictionary:

- Definitely store your data as a dictionary
 - Why? Makes it easier to retrieve items
- Do not use the name or title as the key
 - Why? Two items (people, movies) have the same name. Three famous people named Michael Jordan.
- This data structure is best when you need to grab one specific item.

`employees["001"]` # instant lookup



Hint: How did I store my employees?

```
employees = {  
  "results": [  
    {  
      "id": "1",  
      "first_name": "Laura",  
      "last_name": "Sutton",  
      "role": "Regional Manager",  
      "region": "Southwest",  
    },  
    {  
      "id": "2",  
      "first_name": "Willow",  
      "last_name": "Madison",  
      "role": "Assistant to the Regional Manager",  
      "region": "Southwest",  
    },  
  ],  
}
```

ROUTES

```
@app.route('/employee/<int:id>')  
def show_employee(id=None):  
    global employees  
  
    employee = employees[id]
```

Using a dictionary wrapping an array of objects is a common pattern:

- This structure is great for iterating -- if you want to render all books on a page, `employees["results"]`
- Although, lookup by ID is clunkier than `employees["1"]`
- This is what you'll see from real APIs

```
{"results": [...], "count": 3, "page": 1}
```



Hint: How did I store my employees?

```
employees = {  
    "results": [  
        {  
            "id": "1",  
            "first_name": "Laura",  
            "last_name": "Sutton",  
            "role": "Regional Manager",  
            "region": "Southwest",  
        },  
        {  
            "id": "2",  
            "first_name": "Willow",  
            "last_name": "Madison",  
            "role": "Assistant to the Regional Manager",  
            "region": "Southwest",  
        },  
    ]  
}
```

ROUTES

```
@app.route('/employee/<int:id>')  
def show_employee(id=None):  
    global employees  
  
    employee = employees[id]
```

Using an array of objects:

- This structure is great for iterating -- if you want to render all books on a page, `employees["results"]`
- Although, lookup by ID is clunkier than `employees["1"]`
- This is best when you need to display or loop through all the data.

```
for employee in employees:  
    print(employee["first_name"])
```



Hint: How did I store my books?

```
# IMPORT DATA
```

```
filename = os.path.join(app.static_folder, 'data.json')
with open(filename) as book_data:
    books = json.load(book_data)
```

```
# static/data.json
```

```
"results": [{
    "id": 2,
    "list_name": "Hardcover Nonfiction",
    "display_name": "Hardcover Nonfiction",
    "title": "A PROMISED LAND",
    "author": "Barack Obama",
    "publisher": "Crown",
}]
```

- You'll need to add import os, json at the top alongside the Flask imports.
- The file goes in the `static/` folder, which is where Flask looks when you use `app.static_folder`.
- Note: this `open()` call runs once when the server starts, so the data is loaded into memory.

```
for book in books:
    print(book["title"])
```



Homework 06: Search Application Functionality & Usability

Warm Up

What dataset will you choose?

You must pick a dataset that someone might want to search. Your dataset needs to have **at least 10 items**. We suggest you create them manually, like the list of local businesses in the log_sales application.

Each data item needs to have multiple properties. At the minimum, this must include the following:



Homework 06: Search Application Functionality & Usability

- Unique ID (e.g. `id: 1`)
- Short title or name (e.g. `Film: "Parasite", NBA Player: "Jayson Tatum"`)
- Media Link (image, video or gif)
 - For images → link to an image URL
 - For large files → use an external link (YouTube, Vimeo, etc.)
 - Do NOT upload large files
- Description (write a real paragraph with at least 4 sentences)
- At least one numeric field (e.g. `Year: 2019; Price: 24.99, etc`)
- A list (array) of related data (e.g. `genres: ["Drama", "Thriller"], popular_drinks: ["Oat Latte", "Cold Brew"], awards: ["All-Star 2022", "All-NBA First Team"], etc`)



Homework 06: Search Application Functionality & Usability

Main

Code AND a YouTube video

What to submit:

1. HW6_UNI.zip: A Flask project containing:
 - `server.py`
 - `templates/` (and the HTML templates you need)
 - `static/` (and any static files you need)
2. A link to a YouTube video showing off the functionality of your site.

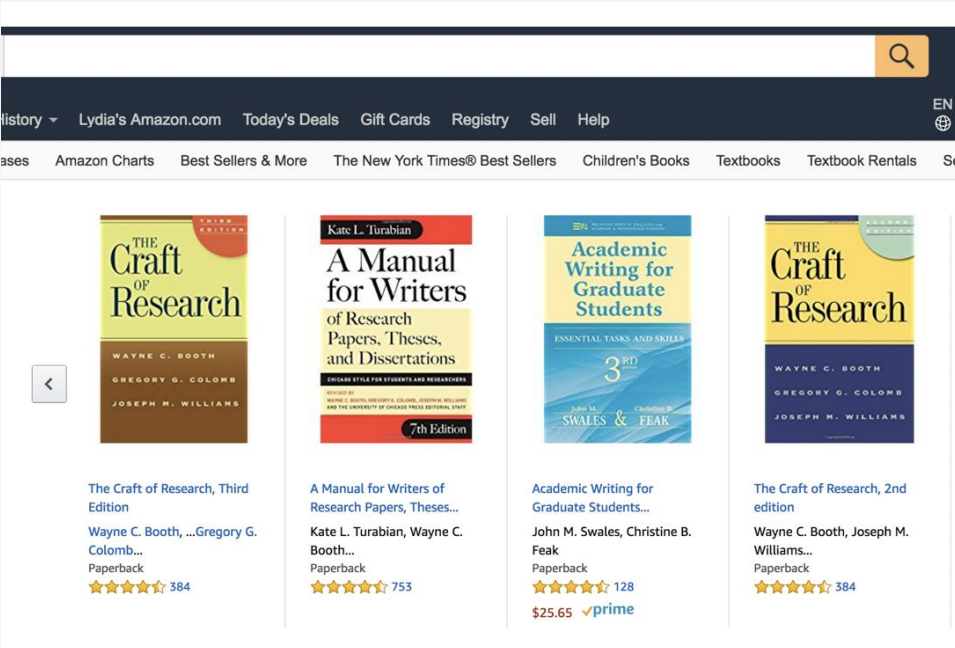
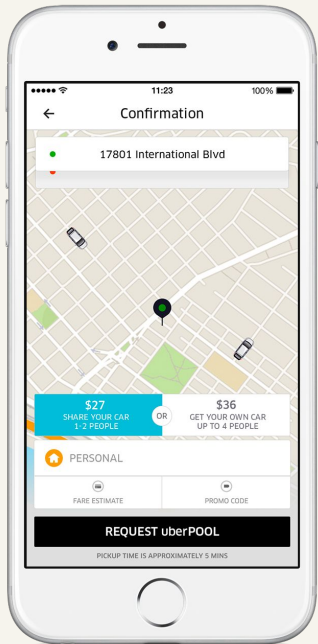
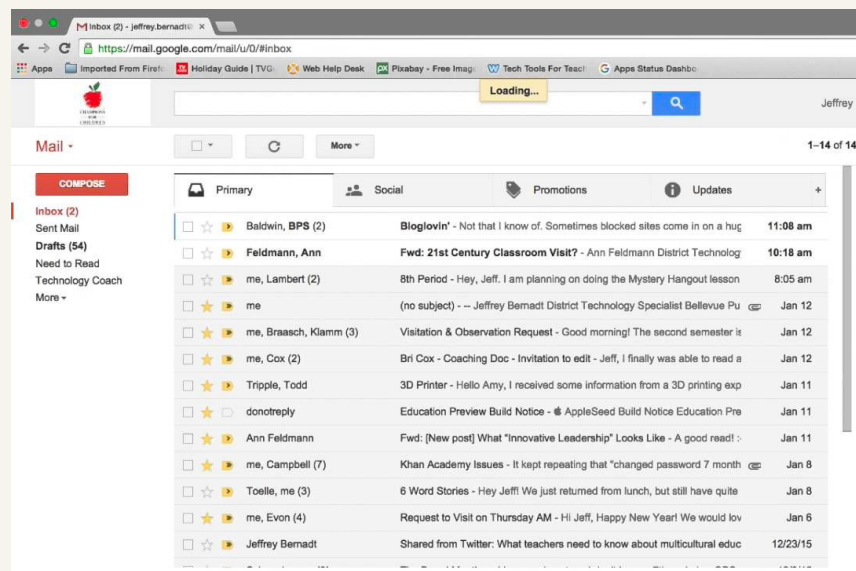


COMS 4170 · SPRING 2026

Summary



The main goal of many websites is to interact with data.



CRUD: Operations for interacting with a database

Create

New Message

To Lee C. Bollinger (bollinger@columbia.edu) ✕ |

From Lydia Chilton <chilton@cs.columbia.edu> ▾

Subject

Hey dude!

 What's on your mind, Lydia?

(No title)
7 - 8pm

Add title

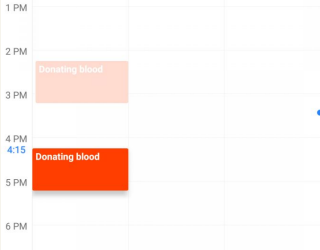
Event Reminder Appointment slots

🕒 Feb 28, 2018 7:00pm - 8:00pm Feb 28, 2018

MORE OPTIONS SAVE

Read

Baldwin, BPS (2)	Bloglovin' - Not that I know of. Sometimes bl
Feldmann, Ann	Fwd: 21st Century Classroom Visit? - Ann
me, Lambert (2)	8th Period - Hey, Jeff. I am planning on doing
me	(no subject) - -- Jeffrey Bernadt District Techr



Delete Post?

Are you sure you want to delete it?

Delete Cancel

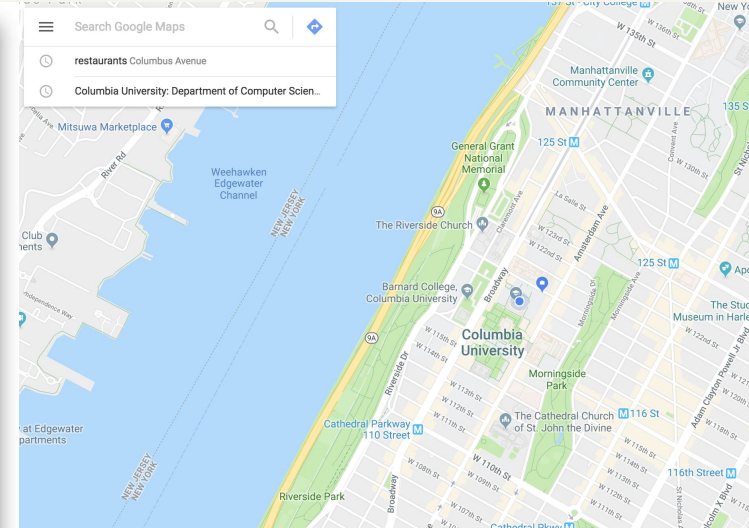
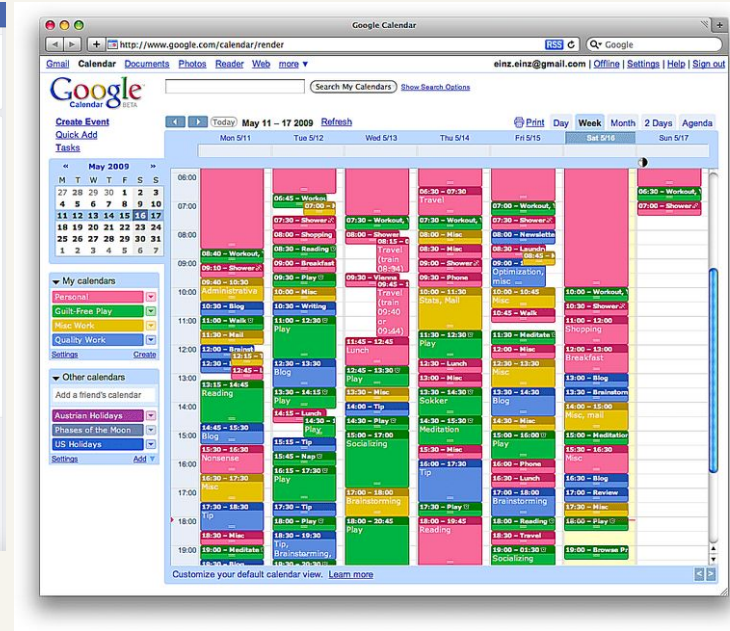
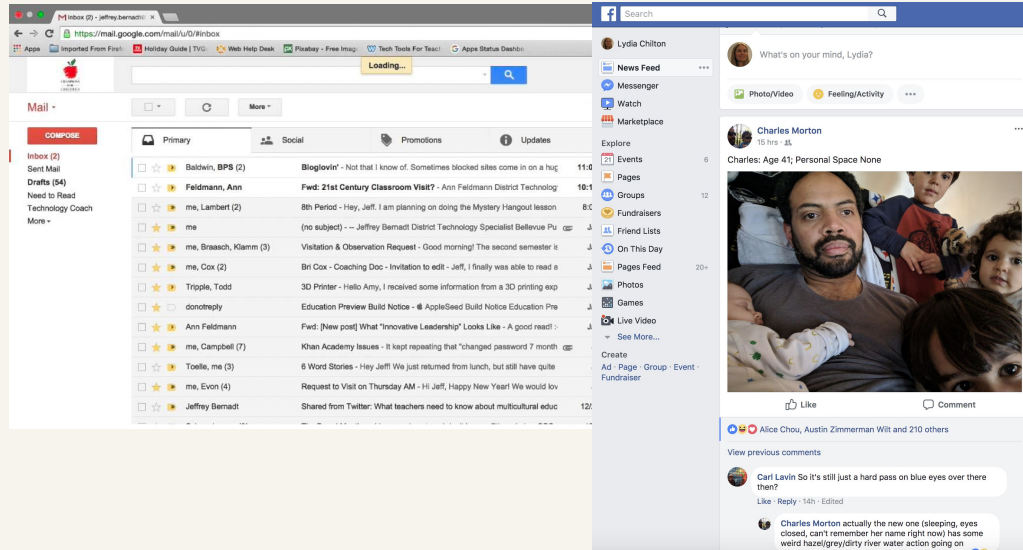
4170

 Delete event

Update

Delete

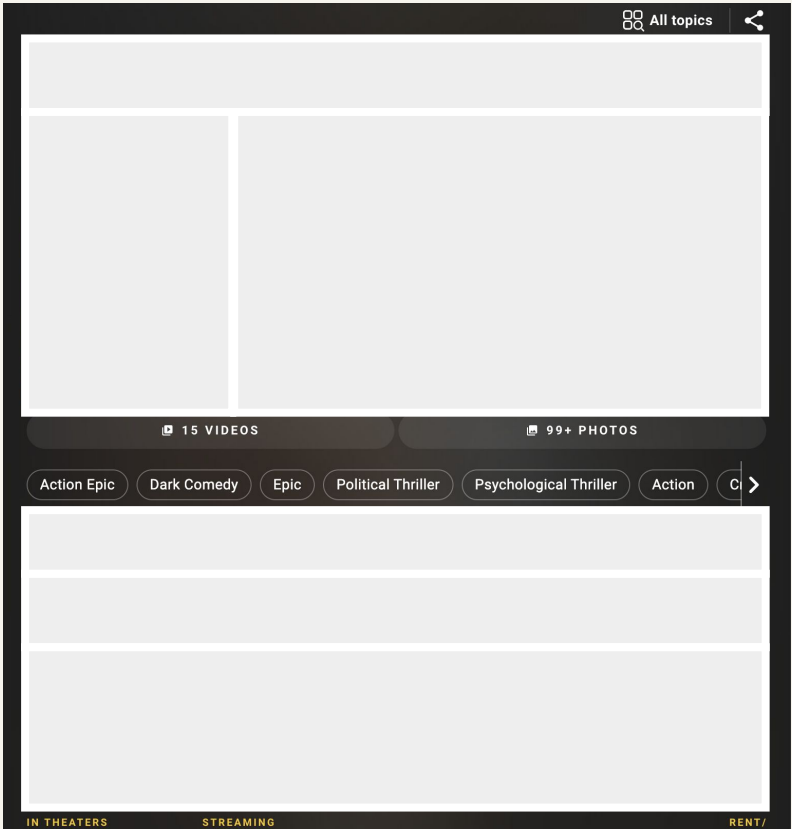
From a back-end perspective. Database-backed websites are very similar.



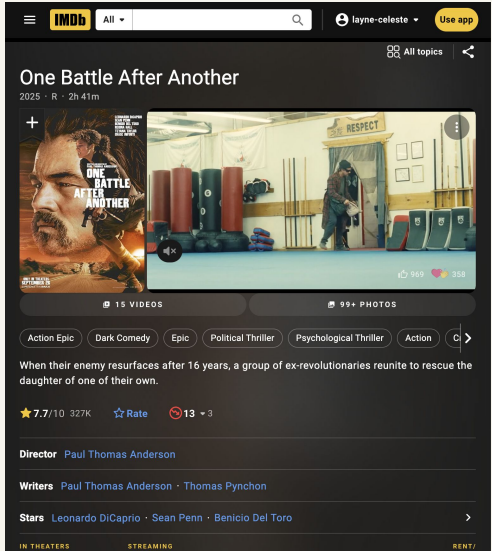
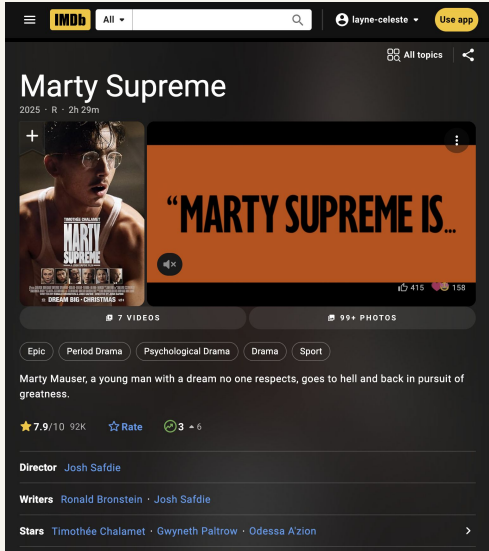
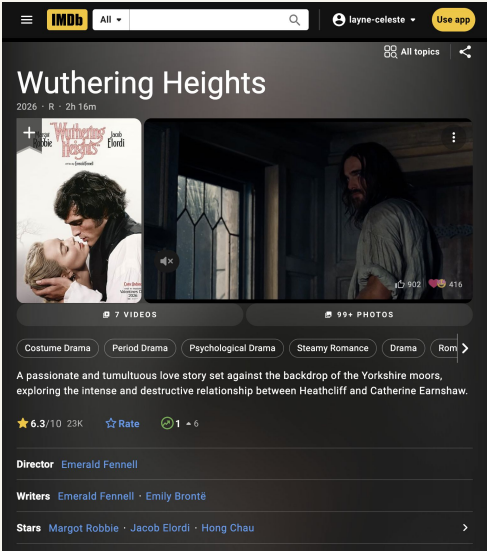
But they serve different data and different information needs



Use templates to dynamically generate pages from data in the database



Title	Plot Summary	Poster	Year
One Battle After Another	When their enemy resurfaces after...	one-battle.jpg	2025
Wuthering Heights	A passionate and tumultuous love...	wuthering.jpg	2026
Marty Supreme	Marty Mauser, a young man with a dream...	marty-supreme.jpg	2025



How to render a template with data

Flask route params

```
# import flask
from flask import Flask, url_for

# import jinja template engine
from flask import render_template

# create an app instance
app = Flask(__name__)

# create a base route
@app.route('/')

# create home method
def home():
    return render_template('index.html')

# create a welcome route
@app.route('/welcome/<name>')

# create welcome method
def welcome(name=None):
    return render_template('welcome.html', name=name)
```

